

Trabajo fin de grado

Clasificación auto-supervisada de imágenes mediante aprendizaje no
contrastivo



Francisco Lerma Martínez

**UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR**



Grado en Ingeniería Informática

TRABAJO FIN DE GRADO

**Clasificación auto-supervisada de imágenes
mediante aprendizaje no contrastivo**

Autor: Francisco Lerma Martínez

**Tutor: Miguel Ángel García
García**

mayo 2023

CLASIFICACIÓN AUTO-SUPERVISADA DE IMÁGENES MEDIANTE APRENDIZAJE NO CONTRASTIVO

AUTOR: Francisco Lerma Martínez
TUTOR: Miguel Ángel García García



Video Processing and Understanding Lab
Escuela Politécnica Superior
Universidad Autónoma de Madrid
Mayo 2023

Trabajo parcialmente financiado por el Ministerio de Ciencia e Innovación del
Gobierno de España bajo el proyecto PID2021-125051OB-I00
(HarvestingVisualData) (2022-2025).



Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la Ley, cualquier forma de reproducción, distribución comunicación pública y transformación de esta obra sin contar con la autorización de los titulares de la propiedad intelectual.

La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. del Código Penal*).

DERECHOS RESERVADOS

© 20 de Mayo de 2023 por UNIVERSIDAD AUTÓNOMA DE MADRID

Francisco Tomás y Valiente, n^o 1 Madrid,

28049

Spain

Francisco Lerma Martínez

Clasificación auto-supervisada de imágenes mediante aprendizaje no contrastivo

Francisco Lerma Martínez

C\ Francisco Tomás y Valiente N^o 11

IMPRESO EN ESPAÑA – PRINTED IN SPAIN

AGRADECIMIENTOS

Me gustaría empezar dando las gracias a mi tutor Miguel Ángel García García por haberme propuesto el tema del trabajo, haberme resuelto todas las dudas que fueron surgiendo y realizarme sugerencias a la hora de redactar los diferentes apartados.

En último lugar, me gustaría dar las gracias a mi padre y a mi hermano por el constante apoyo tanto emocional como económico para que pudiera seguir cursando mis estudios sin ninguna otra preocupación.

Resumen

En la actualidad, el aprendizaje automático es una herramienta esencial en la realización de muchas tareas como pueden ser la visión artificial, los asistentes de voz o la conducción autónoma. Esto se debe al gran desarrollo tecnológico del hardware de la última década y a la proliferación de los datos gracias a Internet. No obstante, la mayoría de los datos que se obtienen en la actualidad no están etiquetados, lo cual imposibilita el aprendizaje supervisado. Etiquetar datos es un proceso manual y costoso que requiere de mucho tiempo y, a veces, de conocimiento profesional. Por lo cual, surge el aprendizaje auto-supervisado que permite entrenar extractores de características capaces de aprender representaciones útiles y robustas de datos no etiquetados. Estos extractores de características preentrenados de manera auto-supervisada pueden ser incluidos posteriormente en otras redes neuronales gracias al *transfer learning*, facilitando la realización de otras tareas supervisadas y reduciendo el número de datos etiquetados necesarios.

En este trabajo, se comienza estudiando en el estado del arte el aprendizaje supervisado mediante redes neuronales convolucionales y el aprendizaje auto-supervisado no contrastivo basado en redes siamesas simples, conocido como *SimSiam*. Este método maximiza la similitud entre dos modificaciones positivas de una misma imagen no etiquetada, sin colapsar en una solución con salida trivial o constante.

El objetivo principal de este trabajo es implementar *SimSiam* en *Pytorch* y preentrenar con este método los extractores de características de las redes neuronales convolucionales *ResNet*, *VGG*, *AlexNet*, *SqueezeNet* y *MobileNet*. Posteriormente, se compararán los resultados obtenidos de la clasificación supervisada con estas redes neuronales convolucionales con los resultados obtenidos de los clasificadores auto-supervisados, construidos sobre los extractores de características preentrenados con *SimSiam*. Las pruebas experimentales se realizarán sobre los conjuntos de datos *Imagenette* e *ImageWoof*, los cuales son subconjuntos de 10 clases de *ImageNet*. Por último, se muestran los resultados y conclusiones obtenidos de los experimentos para cada red neuronal convolucional, así como la aptitud de estas redes neuronales para este tipo de aprendizaje auto-supervisado.

PALABRAS CLAVE

Aprendizaje supervisado, aprendizaje auto-supervisado no contrastivo, redes siamesas, *SimSiam*,

redes neuronales convolucionales, *ResNet*, *VGG*, *AlexNet*, *SqueezeNet*, *MobileNet*.

ABSTRACT

Nowadays, machine learning is an essential tool in the performance of many tasks such as artificial vision, voice assistants or autonomous driving. This is due to the great technological development in hardware in the last decade and the proliferation of data thanks to the Internet. However, most of this data is not labelled, making supervised learning not suitable. Labelling data is a manual and costly task that requires a lot of time and, sometimes, professional knowledge. Hence, self-supervised learning arises, which allows training feature extractors capable to learn useful and robust representations of unlabelled data. These self-supervised feature extractors can be included in other neural networks thanks to transfer learning, facilitating the performance of other supervised tasks and reducing the amount of labelled data required.

In this paper, we study supervised learning using convolutional neural networks and we study non-contrastive self-supervised learning based on simple Siamese networks, known as *SimSiam*. This method maximizes the similarity between two positive augmentations of one unlabelled image, without collapsing to constant or trivial outputs.

The main objective of this paper is to implement *SimSiam* in *Pytorch* and to pre-train the feature extractors of the convolutional neural networks *ResNet*, *VGG*, *AlexNet*, *SqueezeNet* y *MobileNet* with this method. Subsequently, the results obtained from the supervised classification with these convolutional neural networks will be compared with the results obtained from the self-supervised classifiers, built on the feature extractors pre-trained with *SimSiam*. Experimental tests will be performed on the *Imagenette* and *ImageWoof* datasets, which are subsets of 10 *ImageNet* classes. Finally, we show the results and conclusions obtained from the experimental tests for each convolutional neural network and which are more suitable for this type of self-supervised learning.

Keywords

Supervised learning, non-contrastive self-supervised learning, Siamese networks, *SimSiam*, convolutional neural networks, *ResNet*, *VGG*, *AlexNet*, *SqueezeNet*, *MobileNet*.

Índice

1	Introducción	1
1.1	Motivación	1
1.2	Objetivos	3
1.3	Estructura del documento	3
2	Estado del arte	5
2.1	Fundamentos de redes neuronales	5
2.1.1	Redes neuronales MLP	5
2.1.2	Descenso del gradiente y <i>backpropagation</i>	8
2.1.3	SGD, <i>batch size</i> , <i>weight decay</i> y <i>momentum</i>	10
2.1.4	<i>Dropout</i> , <i>data augmentation</i> y <i>batch normalization</i>	11
2.2	<i>ImageNet</i>	12
2.3	Redes neuronales convolucionales.....	13
2.3.1	Capa convolucional	13
2.3.2	Capa de <i>pooling</i>	14
2.3.3	<i>AlexNet</i>	15
2.3.4	VGG	15
2.3.5	<i>ResNet</i>	15
2.3.6	<i>SqueezeNet</i>	17
2.3.7	<i>MobileNet</i>	17
2.4	Aprendizaje supervisado y no supervisado.....	18
2.5	Aprendizaje auto-supervisado.....	19
2.5.1	<i>SimSiam</i>	20
3	Desarrollo y diseño	23
3.1	Entorno de desarrollo.....	23
3.2	<i>Imagenette</i> e <i>ImageWoof</i>	24
3.3	Implementación de la clasificación supervisada.....	25
3.4	Implementación de <i>SimSiam</i>	26
3.5	Implementación de la clasificación auto-supervisada.....	28
4	Resultados	29
4.1	Resultados de la clasificación supervisada.....	29
4.2	Resultados del preentrenamiento con <i>SimSiam</i>	31
4.3	Resultados de la clasificación auto-supervisada.....	34

4.3.1 Comparativa de la clasificación supervisada y autosupervisada.....	37
5 Conclusiones	39
5.1 Trabajo futuro.....	40
Bibliografía	43
Apéndices	45
A Código desarrollado	47
A.1 Clase <i>SimSiam</i> adaptada.....	47
A.2 Código <i>data augmentation</i>	50
A.3 Instalación.....	52

LISTAS

Lista de códigos

A.1	Código de la clase del proyector MLP de <i>SimSiam</i>	48
A.2	Código de la clase del predictor MLP de <i>SimSiam</i>	48
A.3	Código de la clase de <i>SimSiam</i>	49
A.4	Código de las transformaciones de las imágenes de entrenamiento supervisado	50
A.5	Código de las transformaciones de las imágenes de validación	50
A.6	Código de las transformaciones de las imágenes de <i>SimSiam</i>	50
A.7	Código de las transformaciones de las imágenes de entrenamiento auto-supervisado .	51
A.8	Comando de instalación de <i>Pytorch</i>	52
A.9	Comando de instalación de <i>Neptune</i>	52

Lista de ecuaciones

2.1	Salida neurona artificial	5
2.2	Propagación hacia adelante	6
2.3	Propagación hacia adelante en forma matricial	7
2.4	<i>Softmax</i>	8
2.5	Error Cuadrático Medio (ECM)	8
2.6	<i>Cross-Entropy (CE)</i>	8
2.7	Descenso del gradiente	9
2.8	<i>Batch Cross-Entropy Loss</i>	10
2.9	<i>Weight decay</i> en el descenso del gradiente	11
2.10	a <i>Momentum</i> en el descenso del gradiente	11
2.10b	Definición de VT	11
2.10	c Definición de V1	11
2.11	<i>Batch Normalization</i>	12
2.12	Función similitud del coseno negativo	21
2.13	Función de coste simétrica de <i>SimSiam</i>	21
2.14	Función de coste simétrica de <i>SimSiam</i> con stopgrad	21

Lista de figuras

2.1	Estructura de una neurona artificial	5
2.2	Estructura del MLP	6
2.3	Funciones sigmoide, tangente hiperbólica y ReLU	7
2.4	Efecto del batch size en el descenso del gradiente	10
2.5	Estructura y salida de una capa convolucional	13
2.6	Salida de una capa de <i>max pooling</i> y <i>average pooling</i>	14
2.7	Arquitectura de AlexNet	15
2.8	Arquitectura de VGG13	16
2.9	Estructura bloque residual	16
2.10	Arquitectura de ResNet34	16
2.11	Arquitectura de SqueezeNet1.1	17
2.12	Estructura del bloque residual de MobileNetv2	18
2.13	Arquitectura de <i>SimSiam</i>	22
3.1	Clases de <i>Imagenette</i>	24
3.2	Clases de <i>ImageWoof</i>	25
4.1	Tiempos de entrenamiento para la clasificación supervisada.	30
4.2	Gráficas de la <i>loss</i> y la <i>accuracy</i> top-1 de la clasificación supervisada de <i>Imagenette</i>	30
4.3	Gráficas de la <i>loss</i> y la <i>accuracy</i> top-1 de la clasificación supervisada de <i>ImageWoof</i>	31
4.4	Tiempos de los preentrenamientos con <i>SimSiam</i>	32
4.5	Gráficas de la <i>loss</i> y la <i>accuracy</i> top-1 del kNN en el preentrenamiento con <i>Imagenette</i>	32
4.6	Gráficas de la <i>loss</i> y la <i>accuracy</i> top-1 del kNN en el preentrenamiento con <i>ImageWoof</i>	32
4.7	Visualización de los vectores de características en 2D de <i>Imagenette</i>	33
4.8	Visualización de los vectores de características en 2D de <i>ImageWoof</i>	34
4.9	Tiempos de entrenamiento para la clasificación auto-supervisada por arquitectura.	35
4.10	<i>Accuracy</i> top-1 (%) de los clasificadores auto-supervisados	36
4.11	Gráficas de la <i>loss</i> y la <i>accuracy</i> de la clasificación auto-supervisada	36
A.1	Figura con las modificaciones a una misma imagen	51

Lista de tablas

3.1	Número de imágenes por clase en <i>Imagenette</i>	24
3.2	Número de imágenes por clase en <i>ImageWoof</i>	25
4.1	<i>Accuracy</i> top-1 y top-5 de la clasificación supervisada de <i>Imagenette</i> e <i>ImageWoof</i>	29
4.2	<i>Accuracy</i> del clasificador kNN durante el preentrenamiento con <i>SimSiam</i>	33

En este Trabajo de Fin de Grado se estudia el **aprendizaje supervisado y auto-supervisado no contrastivo** de redes neuronales convolucionales, en adelante **CNNs** por sus siglas en inglés, para la clasificación de imágenes. Concretamente, se estudia e implementa el método de entrenamiento auto-supervisado no contrastivo llamado **SimSiam**, véase [1], con diferentes CNNs y se compara las métricas obtenidas de esta manera con las de las CNNs entrenadas de manera supervisada.

1.1. Motivación

A pesar de parecer un campo actual, la teoría sobre redes neuronales empezó en 1943 con la teoría de la neurona artificial por McCulloch-Pitts, véase [2]. De aquí nace la idea de crear redes neuronales artificiales para simular las neuronas del cerebro humano, el cual es capaz de aprender por sí mismo a partir de diferentes estímulos, y poder replicar el aprendizaje de manera automática. Cabe destacar en 1957 la creación del perceptrón por Rosenblatt [3], capaz de resolver problemas de clasificación lineal binaria. Más tarde en 1965 se desarrollaría el **perceptrón multicapa (MLP)**, el método de **backpropagation** responsable del aprendizaje y de minimizar el error de las redes y en 1989 la implementación de las CNNs capaces de extraer características de los datos de entrenamiento [4]. Pese a todo ello, el limitado hardware y capacidad de computación de la época y la poca capacidad para extraer y clasificar datos limitaron su utilización a casos pequeños de pocas capas y a los libros de texto. Por ejemplo, en 2008 las redes neuronales no se encontraban ni en el top 10 de los algoritmos más usados para la minería de datos, véase [5]. En esta última década, gracias al gran volumen de datos clasificados del que se dispone gracias a Internet y al desarrollo exponencial del hardware y del poder de computación se ha producido un gran desarrollo en el campo. Ejemplos de gran actualidad serían los modelos generativos de lenguaje o los generadores de imágenes a partir de texto.

Uno de los principales usos del aprendizaje automático es la tarea de reconocimiento o clasificación de imágenes, en la cual nos centraremos en este trabajo. Esta tarea consiste en que, dada una imagen a la red neuronal con la que no ha sido entrenada, este sea capaz de decirnos que clase de imagen es (si es un gato o un perro, por ejemplo) de entre un conjunto de clases de imágenes dadas con las

que ha sido entrenada. Por ejemplo, es de vital importancia en el diagnóstico de enfermedades como los melanomas [6], en el reconocimiento facial [7], etc.

A pesar de que la clasificación de imágenes se pudiera hacer con redes neuronales básicas como el MLP, la llegada e implementación de las CNNs en 1989, véase [8], supuso un cambio en el paradigma. Estas redes disponen de capas convolucionales, que pueden ser entendidas como filtros, que se aplican a diferentes partes de la imagen. Estos filtros son capaces de extraer características invariantes de las imágenes como pueden ser líneas verticales, horizontales, esquinas, etc. Estas capas permiten que la red reconozca los patrones que se repiten en las imágenes independientemente de si se encuentran en una posición u otra de la imagen. Estos filtros comparten los pesos, de manera que el número de parámetros se reduce considerablemente reduciendo el costo de la computación de estas redes, a diferencia del MLP que necesitará muchos más parámetros para el mismo resultado.

Esta clasificación se realiza principalmente de manera supervisada, esto quiere decir que es necesario entrenar previamente a las redes con imágenes etiquetadas para que sepan a qué clase pertenece cada imagen y poder disminuir su error. Por ejemplo, si se quiere que una red clasifique diferentes tipos de ropa, se le tendrá que decir a la red durante el entrenamiento que imagen es un zapato, cual es una camiseta, etc. Esta tarea de etiquetar es muy costosa, ya que es un trabajo manual humano que se tiene que realizar para cada imagen y normalmente se necesitan miles o incluso millones de imágenes para alcanzar una *accuracy* o precisión aceptable. Además, en muchos campos, como puede ser el médico, se necesitan a profesionales para etiquetar las imágenes, incrementando el coste y el tiempo necesarios [6].

A pesar de disponer de grandes bases de datos anotadas con más de un millón de imágenes etiquetadas como puede ser *ImageNet*, véase [9], la obtención de bases de datos etiquetadas sigue siendo un reto actual. Por ello, surge la necesidad de técnicas de aprendizaje no supervisado donde no se necesiten que los datos estén etiquetados. En nuestro caso se estudia el aprendizaje auto-supervisado, el cual intenta obtener representaciones útiles de datos no etiquetados. Este aprendizaje se puede considerar que se encuentra entre el supervisado y no supervisado, al no necesitar datos etiquetados por el ser humano, ya que él mismo genera sus etiquetas de manera automática. A partir de estas etiquetas generadas automáticamente se pueden entrenar las redes para ciertas tareas pretexto como pueden ser predecir la rotación de una imagen [10], rellenar una imagen hueca con un parche o decir la posición relativa de un recorte [11]. Estas redes podrían ser incluidas mediante *transfer learning* en otras redes para la realización de otra tarea supervisada posterior, facilitando el entrenamiento, aumentando el rendimiento y reduciendo el número de datos etiquetados necesarios en el entrenamiento, véase [12].

1.2. Objetivos

Como se ha comentado al principio, el principal objetivo del trabajo es comparar las métricas obtenidas de las principales arquitecturas de redes CNNs entrenadas de manera supervisada y de manera auto-supervisada por **SimSiam**. A continuación, se enumeran los objetivos que se quieren alcanzar de manera cronológica:

- O-1.**– Estudiar las redes neuronales convolucionales y las distintas arquitecturas más utilizadas en la actualidad, concretamente **AlexNet**, **ResNet**, **MobileNet**, **VGG** y **SqueezeNet**.
- O-2.**– Estudiar el aprendizaje auto-supervisado y más concretamente el método de aprendizaje auto-supervisado no contrastivo **SimSiam**.
- O-3.**– Implementar en **Pytorch** las diferentes redes convolucionales anteriormente nombradas y entrenarlas de manera supervisada para la clasificación de imágenes.
- O-4.**– Implementar en **Pytorch** **SimSiam** y preentrenar las capas extractoras de características de las redes convolucionales anteriormente nombradas con este método.
- O-5.**– Comparar dentro de las redes entrenadas de manera auto-supervisada que arquitecturas realizan una **mejor extracción de características**.
- O-6.**– Implementar **clasificadores auto-supervisados** con los extractores de características preentrenados con **SimSiam** y comparar los resultados con los clasificadores supervisados.

1.3. Estructura del documento

El documento se estructura en 5 capítulos, siendo este el primer capítulo. En el capítulo 2 se trata el **estado del arte**. Se hace una breve introducción a la teoría de las redes neuronales, concretamente la de las redes neuronales *multilayer perceptron* (MLP) y CNNs. Dentro de las CNNs, se explican sus diferentes capas y se presentan las arquitecturas que se usarán en el desarrollo del proyecto. Posteriormente, se introducen los distintos tipos de aprendizaje. Se trata con mayor profundidad el aprendizaje auto-supervisado no contrastivo mediante **SimSiam** que se implementa en los experimentos.

En el capítulo 3 se trata el **desarrollo y diseño**. Se explica el entorno de desarrollo usado, la implementación de las distintas redes para cada entrenamiento, así como los diferentes hiperparámetros de cada red para cada tipo de entrenamiento, las bases de datos utilizadas y el preprocesamiento de los datos.

En el capítulo 4 se muestran y discuten los principales **resultados** de los experimentos. En la Sección 4.1 se muestran y discuten los resultados y métricas obtenidos para la clasificación supervisada de las CNNs. En la Sección 4.2 se muestran y discuten los resultados y métricas de la etapa de preentrenamiento mediante **SimSiam** de las CNNs. Por último, en la Sección 4.3 se mostrarán los resultados y métricas obtenidos para la clasificación auto-supervisada al entrenar el clasificador de las CNNs auto-supervisadas, y se compararán con los obtenidos de manera supervisada.

En el capítulo 5 se presentan las **conclusiones** y se resume el trabajo realizado y los objetivos conseguidos, además se discute el posible trabajo a futuro que se podría realizar. Finalmente se incluye la bibliografía y un apéndice A donde se puede encontrar el repositorio con el código desarrollado y las distintas modificaciones realizadas a las imágenes.

En este capítulo se explicarán los fundamentos principales sobre las redes neuronales, las redes neuronales convolucionales (CNNs), los diferentes tipos de aprendizaje y la base de datos *Imagenet*. Dentro de los tipos de aprendizajes, se detalla con mayor profundidad el aprendizaje auto-supervisado y el método no contrastivo *SimSiam*.

2.1. Fundamentos de redes neuronales

2.1.1. Redes neuronales MLP

Primero vamos a explicar el caso más simple de red neuronal, la formada por una sola **neurona artificial**, véase la figura 2.1.

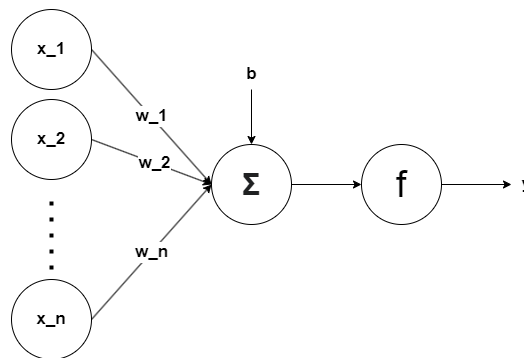


Figura 2.1: Estructura de una neurona artificial.

La entrada de la neurona tendrá el tamaño de la dimensión de los datos $x = (x_1, \dots, x_n) \in \mathbb{R}^n$. La salida $y \in \mathbb{R}$ de esta neurona vendrá dada por

$$y = f\left(\sum_{j=1}^n w_j x_j + b\right). \quad (2.1)$$

Donde $w = (w_1, \dots, w_n) \in \mathbb{R}^n$ es el vector que contiene los **pesos** que conectan las entradas con la

salida de la neurona, $b \in \mathbb{R}$ es el **bias** y f es una función no lineal escalar que suele llamarse **función de activación**. Con esta neurona, debido a su simplicidad, se podrá clasificar como máximo 2 clases.

La principal utilidad del *bias* es la de poder desplazar la función de activación lateralmente, y de esta manera no quedarnos restringidos a un espacio. En cambio, la función de activación es utilizada para añadir no linealidad en el sistema, ya que si no estaríamos restringidos a solo poder resolver problemas lineales. Esta función de activación necesita ser diferenciable en casi todo punto para poder ser usada en el algoritmo *backpropagation* (véase la Subsección 2.1.2). Veamos el caso de una red neural con diferentes capas, también llamada **perceptrón multicapa (MLP)** [4], donde cada una de las capas está formada por diferentes neuronas que están conectas con todas las neuronas de la siguiente capa, véase la figura 2.2. A estas capas también se les suele llamar **fully connected (FC)**.

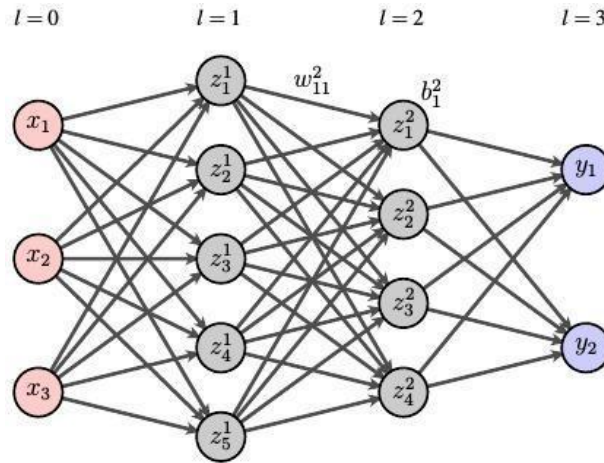


Figura 2.2: Estructura de un perceptrón multicapa con 3 entradas, 2 capas ocultas FC de 5 y 4 neuronas y una capa de salida con 2 neuronas. *Extraído de la American Mathematical Society* ¹.

En este caso la red estará formada por las entradas, las capas ocultas y una capa de salida. Como en el caso anterior $x \in \mathbb{R}^n$ será el vector de entrada de la red e $y \in \mathbb{R}^m$ la salida de la red, donde m será el número de clases que se quiere clasificar. Se denotará a l como la profundidad de la capa en la que nos encontramos, empezando con $l = 0$ las entradas. Se llamará $w_{ij}^{[l]}$ al peso que conecta la j -ésima neurona de la capa $l - 1$ con la i -ésima neurona de la capa l , $b_i^{[l]}$ al *bias* de la neurona i -ésima de la capa l , $f^{[l]}$ a la función de activación de la neurona i -ésima de la capa l y $k^{[l]}$ al número de neuronas i de la capa l . De manera que la salida $z_i^{[l]}$ de la i -ésima neurona de la capa l vendrá dada por

$$z_i^{[l]} = f^{[l]} \left(\sum_{j=1}^{k^{[l-1]}} w_{ij}^{[l]} z_j^{[l-1]} + b_i^{[l]} \right). \quad (2.2)$$

Si la red tiene profundidad L la j -ésima salida de la red vendrá dada por $y_j = z_j^{[L]}$ y las entradas ¹Austin D.,

por $z_j^{[l]} = x$. Si se toma la matriz $W^{[l]} \in \mathbb{R}^{k_{[l-1]} \times k_{[l]}}$ que la cual tiene de elementos a los pesos w

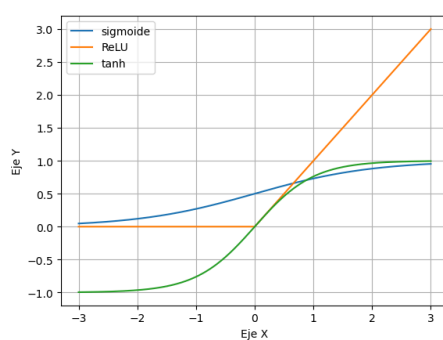
conectan la neuronas de la capa $l - 1$ con la l , los vectores $z^{[l]}, b^{[l]} \in \mathbb{R}^{k_{[l]}}$ las salidas y *bias* de las neuronas de la capa l y la función $f^{[l]} = (f^{[l]}, \dots, f^{[l]})$, se puede usar la siguiente expresión matricial equivalente a (2.2) tal que

$$z^{[l]} = f^{[l]}(W^{[l]}z^{[l-1]} + b^{[l]}). \quad (2.3)$$

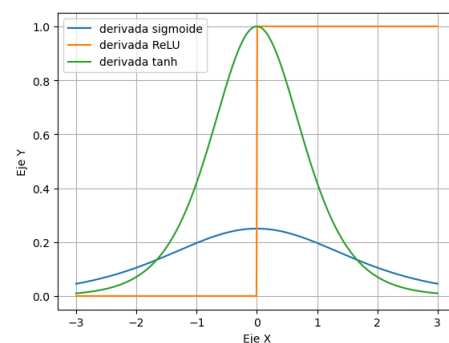
Esta es la forma en la que los ordenadores calculan realmente las salidas debido a la optimización de la multiplicación de matrices. A partir de (2.3) se puede calcular las salidas de todas las neuronas a partir de la entrada, hasta poder obtener las salidas de la última capa. A esto último se le denomina **propagación hacia adelante**.

Como se ha visto, cada neurona tiene su función de activación, aunque de manera general esta suele ser la misma para todas las neuronas, a excepción de las neuronas de la capa de salida. Las funciones de activación más famosas son [4]:

- 1.- **Sigmoide** $\sigma(x) = \frac{1}{1+e^{-x}}$ Es una de las más populares ya que su imagen va de 0 a 1, lo cual nos sirve para expresar probabilidades y también a que se puede calcular su derivada a partir de ella misma ya que $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.
- 2.- **Tangente hiperbólica** $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Su imagen es $(-1, 1)$. Su derivada también se calcula a partir de ella misma ya que $\tanh'(x) = 1 - \tanh^2(x)$.
- 3.- **ReLU (rectified linear unit)** $\text{ReLU}(x) = \max(0, x)$. Su derivada es uno siempre que x sea positiva y cero si no. No se suele usar en las capas de salida.



(a) Funciones de activación



(b) Derivadas de las funciones de activación

Figura 2.3: Funciones sigmoide, tangente hiperbólica y ReLU con sus correspondientes derivadas.

Actualmente la más usada es ReLU ya que es la más eficiente y rápida de calcular al no tener que realizar exponenciales. Además, en la práctica se ha demostrado que las redes entrenadas con ReLU convergen más rápido que con las otras funciones y por tanto se tarda menos en entrenarlas,

véase [13]. Además la sigmoide y tangente tienen derivadas menores que 1 en la mayoría o casi

todo su dominio y cercanas a 0 cuando la x es muy grande (véase la figura 2.3(b)), lo que provoca el problema conocido como *vanishing gradients*. Esto produce que, en la actualidad, al tener tantas capas, estos valores cercanos a cero se acumulen en el algoritmo de *backpropagation* y provocan que el gradiente tienda a cero. Esto no ocurre con ReLU ya que su derivada es uno si x es positiva.

2.1.2. Descenso del gradiente y *backpropagation*

El principal objetivo de las redes neuronales será entrenar los pesos/*bias* de sus neuronas, para que al hacer la propagación hacia adelante la salida de la red coincida con la etiqueta que se espera del dato de entrada. Al inicio la red está inicializada con pesos aleatorios. Si en una capa se usa la función de activación sigmoide o *tanh* se realiza una inicialización *Xavier* [14] y si se usa ReLU se usa la inicialización *He* [15]. En el caso de una clasificación multiclase, las etiquetas serán vectores de ceros con un uno en la posición de la clase a la que pertenezca el dato. De manera que, si se le pasa un dato de la tercera clase, se quiere que la tercera neurona de la salida de la red nos devuelva un uno y el resto ceros. Normalmente la predicción será la clase de la neurona que mayor probabilidad nos devuelva, y por ello se quiere que las salidas de las neuronas estén entre 0 y 1. Para lograr ello se aplica la función **softmax** a la capa de salida definida como

$$e^{z_j} \quad \frac{e^{z_i}}{\sum e^{z_i}}, \quad (2.4)$$

donde z es el vector con las salidas de la red y z_i es la salida de la i -ésima neurona sobre la que se aplica *softmax*. El numerador convierte en un número positivo cada salida al aplicar la exponencial, y después el denominador lo normaliza para que se encuentre entre 0 y 1 y sea una probabilidad.

Para saber si una red es eficaz, se define una **función de coste**, también llamada **loss**, diferenciable que nos diga cómo de incorrecta es la salida. Cuanto menor sea el valor de la función mejor será nuestro clasificador. Por tanto, el principal objetivo será encontrar los parámetros (los pesos y *bias*) que minimicen esa función. Para problemas de regresión la más popular es la función de **Error Cuadrático Medio (ECM)** que se define como

$$ECM = \frac{1}{N} \sum_{j=1}^N (\hat{y}_j - y_j)^2. \quad (2.5)$$

Para el caso de clasificación multiclase la función de coste más popular es la **Cross-Entropy (CE)** **loss** que se define como

$$EC = - \sum_{j=1}^N [y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)]. \quad (2.6)$$

Donde el vector N-dimensional $\hat{y}$ es el vector con la salida de la red, en el caso de (2.6) después de haberle aplicado *softmax* (2.4), e y es el vector con la etiqueta del dato de entrenamiento correspon-

diente. La función *Cross-Entropy* (2.6) tiene como característica que castiga el fallo mucho más que los aciertos. Esto es debido al logaritmo, ya que si la red falla la etiqueta es diferente a la predicción $y_j \neq \hat{y}_j$ y se tienen valores cercanos al $\log(\epsilon)$ que es un número muy grande si ϵ es muy cercano a cero. Esto provocará que los pesos se actualicen como veremos a continuación. Si la red acierta, entonces $y_j \approx \hat{y}_j$ y la CE será el $\log(1 - \epsilon)$ que es casi cero y los parámetros no sufrirán casi cambios.

El siguiente paso será intentar alcanzar el mínimo de la función de coste y es donde entra en juego el algoritmo del **descenso del gradiente**. El gradiente de una función es un campo vectorial formado por las derivadas parciales y el cual si se aplica en un punto nos devuelve la dirección de máximo crecimiento. Para ello se necesita que la función sea diferenciable (las funciones de coste lo son). Por lo tanto, si se toma la dirección contraria negándolo se obtendrá la dirección de mayor descenso. Si se toma un pequeño paso en esa dirección descenderemos y nos iremos acercando a un mínimo local. Si de manera reiterativa se vuelve a calcular el gradiente en el nuevo punto obtenido y se vuelve a tomar un paso en la dirección opuesta, nos iremos acercando a un entorno cercano a un punto mínimo local. De esta manera se obtiene que para minimizar el coste se tienen que calcular y actualizar reiteradamente los pesos y *bias* (ya que son los parámetros o puntos sobre los que depende y_j y por tanto la función coste) [4] en la dirección contraria al gradiente tal que:

$$w^{[l]}_t, b^{[l]}_t = w^{[l]}_{t-1}, b^{[l]}_{t-1} - \gamma \nabla C(w^{[l]}_{t-1}, b^{[l]}_{t-1}) \quad \text{con } t = 1, 2, \dots \quad (2.7)$$

$$w^{[l]}_t, b^{[l]}_t = w^{[l]}_{t-1}, b^{[l]}_{t-1} - \gamma \nabla C(w^{[l]}_{t-1}, b^{[l]}_{t-1}) \quad t = 1, 2, \dots$$

donde ∇C es el gradiente de la función coste, $w^{[l]}_t$ es el vector con los pesos que unen la capa $l - 1$ con la l en la iteración t y $b^{[l]}_t$ es el vector con los *bias* de la capa l en la iteración t . El hiperparámetro γ se llama **learning rate (lr)** y determina el tamaño del paso. Este hiperparámetro es el más importante a la hora de entrenar, ya que si es muy grande se saltará mucho en la función y no se llegará al mínimo, y si es muy pequeño se podrían necesitar más iteraciones para llegar al mínimo o quedarse corto. La primera iteración $w^{[l]}_0$ y $b^{[l]}_0$ serán los pesos y *bias* con los que se inicializó la red. En cada iteración

$$w^{[l]}_0, b^{[l]}_0$$

debería mejorar la predicción de la red y disminuir la función coste. Una de las principales desventajas de este método es que se puede alcanzar un mínimo local en vez del global.

Para el descenso del gradiente se necesita saber cómo calcular ∇C , pero las redes tienen de miles a millones de pesos y *bias*, lo que complica la tarea. Aquí es donde entra en juego el algoritmo **backpropagation**. Este algoritmo hace uso de la regla de la cadena de las derivadas, debido a que las derivadas parciales respecto de los pesos/*bias* de una capa oculta dependen de los pesos/*bias*

de las capas posteriores. De esta manera primero se calculan las derivadas parciales respecto a los pesos/*bias* de la capa de salida, las cuales se obtienen de manera inmediata al conocer la salida, la derivada de la función coste y las inicializaciones de los pesos/*bias*. Después se calculan las derivadas parciales respecto a los pesos/*bias* de la capa anterior gracias a la regla de la cadena, la cual nos convierte la derivada parcial en un producto de factores y derivadas que ya se conocen de la capa anterior. Este algoritmo se realiza reiteradamente, hasta obtener las derivadas parciales respecto a los parámetros de todas las capas. Para una ampliación sobre la teoría se remite a [4].

2.1.3. SGD, batch size, weight decay y momentum

Uno de los hiperparámetros que hay que definir para toda red es el tamaño del lote también llamado **batch size**. El **batch size** es el número de datos de entrenamiento del lote o **batch** con el que se entrena a la red antes de actualizar sus parámetros con el descenso del gradiente. Se llama **Mini Batch Stochastic Gradient Descent (SGD)** al optimizador o método que utiliza un determinado **batch size** y el algoritmo de descenso del gradiente para actualizar a la red. Si el **batch size** es mayor que 1 se usa como función de coste la ponderación de los costes de los datos del lote [16]. Si se tiene un **batch size** M habría que actualizar la ecuación de la **Cross-Entropy loss** (2.6) tal que:

$$\sum_{i=1}^M \sum_{j=1}^N [y_{ij} \log(\hat{y}_{ij}) + (1 - y_{ij}) \log(1 - \hat{y}_{ij})], \quad (2.8)$$

donde $\hat{y}_{ij}$ es la salida de j -ésima neurona de la red para el i -ésimo dato del lote e y_i es el vector con la etiqueta del i -ésimo dato del lote.

Si se utiliza un **batch size** muy grande se necesita mucha memoria para poder almacenar el conjunto de datos, pero nos asegura que la dirección del gradiente minimiza el error para todo el conjunto de datos, a cambio de poder actualizar la red menos veces por iteración completa del conjunto de datos de entrenamiento. Si el **batch size** es más pequeño se actualiza más la red por iteración completa del conjunto de datos y la red convergerá más rápido, pero no nos asegura que el descenso del gradiente produzca la mejor minimización del coste, véase la figura 2.4. Además, a mayor **batch size**, mayor **learning rate** se necesitará, véase [17]. Al hiperparámetro que define el número de veces o ciclos que se recorrerá el conjunto de datos de entrenamiento se llama **época**.

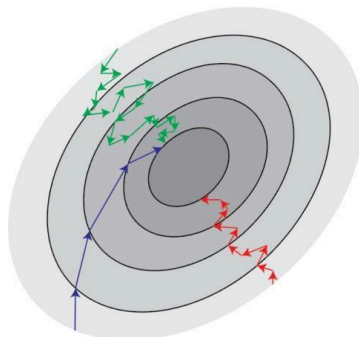


Figura 2.4: Efecto del batch size en el descenso del gradiente. En verde un **batch size** muy pequeño de 1, en rojo uno mediano y en azul si coincide con el tamaño del conjunto de datos de entrenamiento. Extraído de la página web de Baeldung².

Otro principal problema de las redes neuronales es el **overfitting**, el cual es el sobreajuste de la

red a los datos de entrenamiento de manera que obtiene una representación casi exacta de ellos.
Esto

² Zvornicanin E., Relation Between Learning Rate and Batch Size. *Baeldung*, <https://www.baeldung.com/cs/learning-rate-batch-size>, 2023.

provocara que la red no aprenda una representación más general de los datos y no sea precisa con datos con los que no ha sido entrenada. Se ha comprobado que pesos más pequeños en las redes las hacen más simples y capaces de aprender representaciones más generales de los datos, disminuyendo el *overfitting* y mejorando la convergencia [18]. Por ello, se creó el método de regularización **weight decay** que penaliza a los pesos muy grandes y favorece a los más pequeños. Esto se consigue añadiendo una penalización proporcional al peso de cada parámetro en el descenso del gradiente (2.7), mediante un parámetro positivo λ que representa la penalización del *weight decay*, tal que:

$$(w^{[l]}, b^{[l]})_t = (w^{[l]}, b^{[l]})_{t-1} - \gamma \nabla C(w^{[l]}, b^{[l]})_{t-1} - \lambda (w^{[l]}, b^{[l]})_{t-1}, \quad (2.9)$$

Una de las desventajas de SGD es que se puede alcanzar un mínimo local. Una de las soluciones a este problema viene dado por el **momentum**. Esta técnica añade en la SGD de una manera promediada las últimas direcciones que se han tomado en las últimas iteraciones en el descenso del gradiente. Esto provoca que se ejerza cierta inercia hacia una dirección de descenso y se puedan esquivar mínimos locales que nos puedan ralentizar o hacer converger el coste a un mínimo local no deseado [19]. Esto se consigue modificando el descenso del gradiente (2.7), donde $0 < \mu < 1$ es el parámetro correspondiente al *momentum*, como

$$(w^{[l]}, b^{[l]})_t = (w^{[l]}, b^{[l]})_{t-1} - \gamma V_t \quad \text{con } t = 1, \dots, \quad (2.10a)$$

$$V_t^{[l]} = \mu V_{t-1}^{[l]} + (1 - \mu) \nabla C(w^{[l]}, b^{[l]})_{t-1} \quad \text{con } t = 2, \dots, \quad (2.10b)$$

$$= \mu V_t + (1 - \mu) \nabla C(w^{[l]}, b^{[l]})_t$$

$$V_1 = \nabla C(w_0^{[l]}, b_0^{[l]}). \quad (2.10c)$$

2.1.4. Dropout, data augmentation y batch normalization

El *overfitting* es uno de los principales problemas al entrenar redes neuronales. Para saber si una red realiza *overfitting*, normalmente se divide el conjunto de datos de entrenamiento en dos conjuntos, uno con el que se entrena y otro sobre el que se valida la red al terminar cada época (de manera que se puede comprobar cómo actúa la red con datos con los que no ha sido entrenada). De manera que se tendrá *overfitting* siempre que la *accuracy* (el número de predicciones correctas entre el número total de predicciones) de la red siga aumentando con los datos de entrenamiento a costa de que empeore la *accuracy* con los datos de validación. Gracias a la validación se seleccionará el modelo que mejor *accuracy* o menor *loss* haya tenido durante la validación a lo largo del entrenamiento.

El *dropout* es otra técnica de regularización empleada en las redes neuronales. Esta técnica consiste en que durante el entrenamiento cada neurona y sus conexiones tiene una probabilidad p de desactivarse. Esta idea previene que las redes dependan en gran medida de las activaciones de unas neuronas concretas para un correcto funcionamiento, y por lo tanto nos reduce el *overfitting* [20].

Otra manera de reducir el *overfitting* es aumentando el tamaño del conjunto de datos de entrenamiento, véase [13]. Si no se disponen de más datos, una manera de poder aumentar el tamaño es realizar cada época copias modificadas o transformaciones de los datos de entrenamiento. Por ejemplo, en el caso de las imágenes se puede realizar un recorte, cambiar la iluminación, rotar la imagen o meter ruido ya que estas modificaciones no afectan en nada a la etiqueta. Esto provocará que en cada época la red se entrene con un conjunto de datos ligeramente diferente, haciendo mucho más complicado que alcance el *overfitting*. A esta técnica se la conoce como ***data augmentation***.

Por último, entrenar redes neuronales es complicado ya que la distribución de los datos de cada capa cambia en cada iteración de SGD, al actualizarse todos los parámetros de las capas anteriores. Esto provoca que para algunos datos, el entrenamiento de la red sea inestable y se tengan que usar *learning rates* pequeños e inicializaciones específicas. El método de ***batch normalization (BN)*** consiste en la normalización de las entradas de las capas de neuronas, haciéndolas tener la misma distribución. Esto hace a la red más estable, reduce la necesidad de regularizar los datos y aumenta la rapidez de la convergencia, haciendo que la red se entrene de manera más rápida, véase [21]. Si X es el vector con las entradas de la capa de la red de todos los datos del *batch*, se define *batch normalization* como

$$BN(X) = \alpha \sqrt{\frac{X - E(X)}{Var(X) + \epsilon + \beta}} \quad (2.11)$$

Donde $E(X)$ es la media, $Var(X)$ la varianza y α y β son parámetros que la red tendrá que aprender. Por último, ϵ es una constante numérica positiva pequeña para evitar la división por cero.

2.2. ImageNet

ImageNet es una base de datos de más de 14 millones de imágenes etiquetadas pertenecientes a alrededor de 20000 clases diferentes, véase [9]. Estas imágenes fueron obtenidas de Internet y fueron etiquetadas manualmente. Debido a la gran cantidad de clases e imágenes etiquetadas, esta base de datos ha sido la más utilizada en la investigación sobre visión artificial.

Desde 2010, *ImageNet* realiza un concurso anual llamado ILSVRC con el objetivo de que distintas arquitecturas de redes neuronales compitan para ver cual consigue mejores resultados para la clasificación y detección de objetos de las imágenes. En este concurso, se usa un subconjunto de 1000 clases diferentes con alrededor de 1, 2 millones de imágenes de entrenamiento, 50000 imágenes de validación y 150000 imágenes de prueba. En 2012, la red convolucional *AlexNet* [13] consiguió el mejor error obtenido hasta esa fecha, gracias al uso de la computación paralela aportado por las tarjetas gráficas (GPUs). Posteriormente sería superada en 2014 por las redes convolucionales muy profundas VGG [22] y en 2015 por las redes residuales profundas llamadas

ResNets [23]. Las redes convolucionales, sus capas y sus distintas arquitecturas más utilizadas se explican en la Sección 2.3.

2.3. Redes neuronales convolucionales

2.3.1. Capa convolucional

La gran innovación de las **redes neuronales convolucionales (CNNs)** son sus capas convolucionales capaces de extraer características invariantes de las imágenes como pueden ser líneas verticales, horizontales o los bordes de una imagen. Esta capa convolucional puede ser entendida como un conjunto de **filtros** o *kernels*, donde cada filtro está formado por un conjunto de capas. Estos filtros se irán desplazando por píxeles cercanos y producirán una nueva matriz como salida, que se llamará mapa o vector de características. Los elementos de este mapa de características serán los productos escalares de estos píxeles con los valores del *kernel*, véase la figura 2.5. Parámetros a destacar:

- 1.- Tamaño del *kernel* ($d \times h \times w$): el tamaño del alto h y ancho w debe ser menor al del mapa de características entrante para que se pueda realizar la convolución. El número de capas o profundidad del *kernel* d dependerá del conjunto de canales de la entrada. Si se tiene una imagen RGB tendremos 3 canales en la imagen y por lo tanto 3 capas del filtro. Si hay más de una capa, el mapa de características de la salida será la suma de las matrices que producirían por separado cada capa del *kernel*. Los pesos de los *kernels* serán igual para toda la imagen, de manera que se necesitan muchos menos parámetros.
- 2.- Número de *kernels* por capa convolucional. Cada *kernel* provocará una salida diferente, por lo que coincide con el número de canales del mapa de características de la salida.
- 3.- **Stride**: es cuanto se mueve el filtro en cada paso a lo largo de una dirección, normalmente de manera vertical y horizontal.
- 4.- **Padding**: es el número de píxeles con valor cero que se añaden en los bordes de la imagen. Su principal función es aumentar el tamaño de la imagen y hacer que el filtro pase más por los bordes, ya el filtro actúa más sobre los elementos del centro. Por defecto es 0.

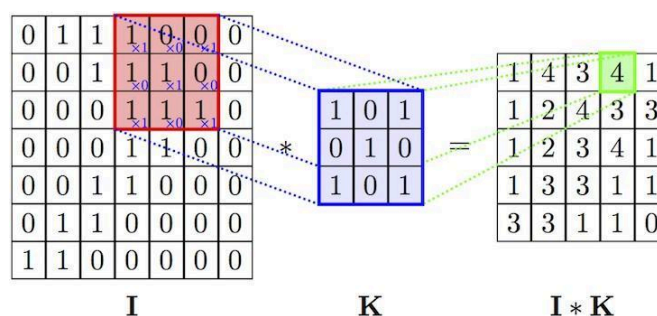


Figura 2.5: Estructura y salida de una capa convolucional. Representa una capa convolucional con *kernel* $1 \times 3 \times 3$ a una imagen 7×7 con stride 1 y padding 0, y su salida. *Extraído de la página web de H. Gasimov: Image Classification With Tensorflow For Beginners*³.

³

H. Gasimov, Image Classification With Tensorflow. <https://www.gasimov.com/blog/image-classification-with-tensorflow-for-beginners/>.

La altura del mapa de características de la salida de la convolución vendrá dado por la fórmula $\frac{W}{S} + 1$, donde W es la altura de la imagen de entrada de la capa, K el tamaño del *kernel*, S el *stride* y P el *padding*. El ancho es análogo, pero cambiando la altura por la anchura. Después de una capa de convolución se aplica una capa o función de activación sobre las salidas, normalmente ReLU [8]. También se suele aplicar *batch normalization* a la salida. Una vez se haya terminado de realizar todas las capas convolucionales, las redes CNNs tienen unas **capas clasificadoras** para resolver el problema de clasificación. Normalmente se usa un MLP como capas clasificadoras, aunque hay casos como *SqueezeNet* que usa una capa convolucional para clasificar [24].

2.3.2. Capa de pooling

Otra de las capas más importantes de las CNNs es la capa de **pooling** encargada de reducir el tamaño de los mapas que las capas convolucionales nos devuelven como salida, eligiendo los elementos más representativos. Esto provoca que se hagan menos operaciones reduciendo el tiempo de entrenamiento y la memoria usada al tener matrices de menor tamaño. Con estas salidas con dimensión reducida se pueden aplicar nuevas capas convolucionales para extraer nuevas características. Además, el número de salidas de estas capas de *pooling* coincide con el número de salidas de las capas convolucionales que las preceden. Al igual que las capas convolucionales, las capas de *pooling* también tienen un *kernel*. Por ello, se definen parámetros similares de tamaño de *kernel*, *stride* y *padding*. Hay dos tipos principales de pooling (ver figura 2.6):

- 1.- *Max pooling*: se escoge como salida el elemento de la matriz de entrada con mayor valor sobre los que se aplica el *kernel*.
- 2.- *Average pooling*: se escoge como salida del filtro la media de los elementos de la matriz de entrada sobre los que se aplica el *kernel*.

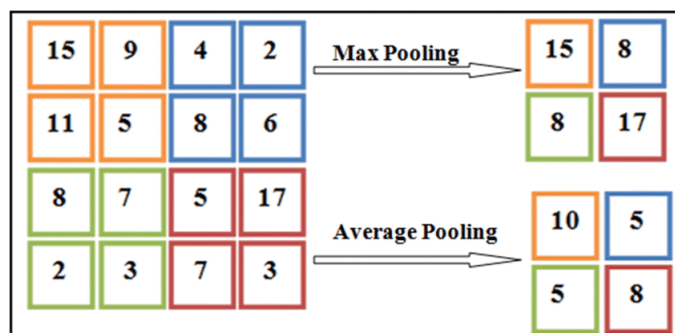


Figura 2.6: Salida de una capa de *max pooling* y *average pooling* sobre una matriz 4×4 con *kernel* de tamaño 2×2 , *stride* 2 y *padding* 0. Extraído de [25].

2.3.3. AlexNet

AlexNet es el nombre de la red convolucional creada por Alex Krizhevsky en 2012 para el problema de clasificación de la base de datos *ImageNet*. El artículo original, véase [13], prueba que la profundidad del modelo es esencial para su rendimiento. Además de introducir nuevos métodos para la paralelización del entrenamiento por GPU, nuevas técnicas de *data augmentation* como la alteración del color de las imágenes, el uso del reciente *dropout* y la función ReLU sobre la entonces frecuente *tanh*. La red está formada por 5 capas convolucionales, 3 capas de *max pool* y 3 capas clasificatorias *fully connected* (FC) (ver figura 2.7). Se aplica ReLU después de cada capa convolucional y FC, exceptuando la salida de la red.

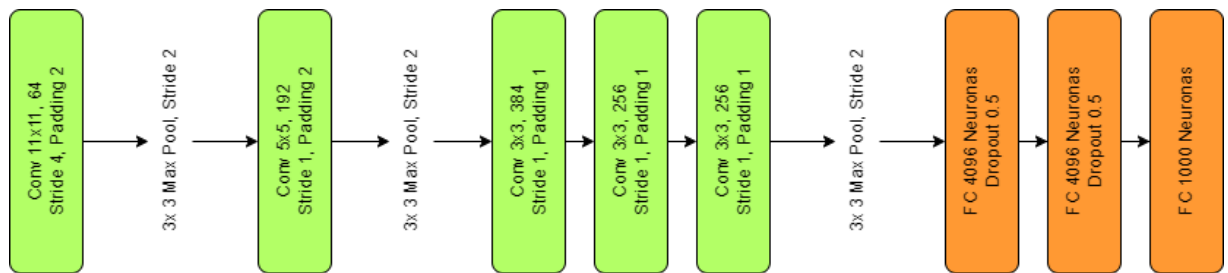


Figura 2.7: Arquitectura de AlexNet. En verde las capas extractoras de características y en naranja las capas clasificatorias.

2.3.4. VGG

Las redes **VGG** son redes convoluciones muy profundas creadas por el departamento de ingeniería *Visual Geometry Group* de la Universidad de Oxford en 2014, y consiguieron la mejor *accuracy* hasta el momento en la clasificación de *ImageNet*. En el artículo original, véase [22], se estudió el incremento en la profundidad de las CNNs y su efecto en el reconocimiento de imágenes. Su principal contribución es que la *accuracy* de una red convolucional que usa *kernels* pequeños de 3×3 aumenta conforme se incrementa el número de capas hasta 16 o 19. Hay diferentes versiones de VGG según el número de capas con pesos, pero todas usan capas convolucionales de tamaño 3×3 con *stride* y *padding* 1, 4 capas de *max pool* de tamaño 2×2 y *stride* 2 y 3 capas clasificatorias FC. Se usa ReLU como función de activación de las capas convolucionales y FC, excepto en la última salida. Además, se aplica un *dropout* con probabilidad 0,5 en las capas FC. La arquitectura con 13 capas con pesos, VGG13, se puede ver en la Figura 2.8.

2.3.5. ResNet

Las **redes residuales profundas (ResNets)** son redes convolucionales creadas por *Microsoft Research* y obtuvieron la mejor *accuracy* en la clasificación de *ImageNet* en 2015, véase [23].

Desde

2012 gracias a AlexNet el estado del arte de las CNNs era crear redes convolucionales más profundas con más capas convolucionales. No obstante, al incrementar el número de capas la *accuracy* finalmente se saturaba y después de un tiempo se reducía. Además, los gradientes pequeños durante la *backpropagation* se acumulan al haber más capas y hacen desaparecer el gradiente (*vanishing gradient*). Para ello *ResNet* introduce bloques residuales con conexiones atajo cada dos capas, las cuales permiten aumentar mucho el número de capas y la *accuracy*, a la vez que se reduce la complejidad. Todas las capas convolucionales de *ResNet* usan *batch normalization* y ReLU y tienen *stride* y *padding* 1, excepto al aumentar la dimensión que se usa un *stride* de 2. La capa *max pool* tiene *kernel* 3×3 , *stride* 2 y *padding* 1. La arquitectura de estos bloques residuales (ver figura 2.9) y de la red (ver figura 2.10) se puede ver para el caso de 34 capas con pesos.

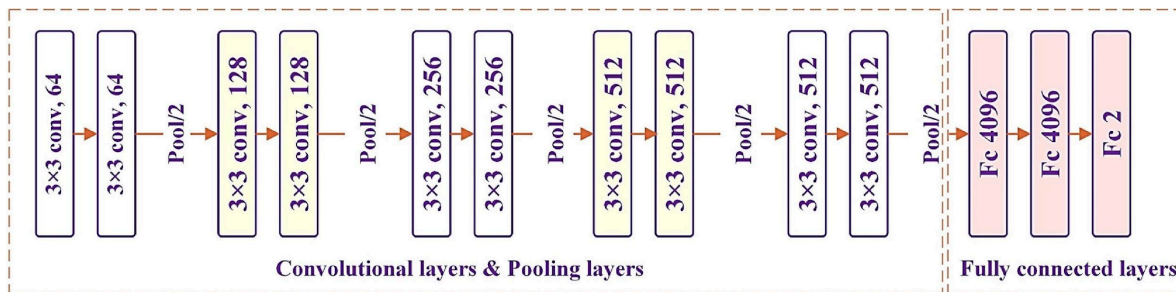


Figura 2.8: Arquitectura de VGG13. El primer rectángulo discontinuo señala la parte extractora de características formada por capas convolucionales y de *max pooling*. El segundo señala las capas FC encargadas de la clasificación. *Extraído de [26]*.

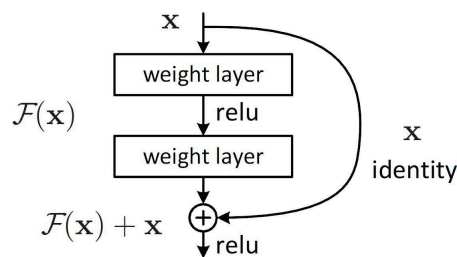


Figura 2.9: Estructura de un bloque residual de Resnet34. La salida del bloque es la suma de la entrada x del bloque con la salida de las dos capas convoluciones $F(x)$ y ReLU. *Extraído de [23]*.

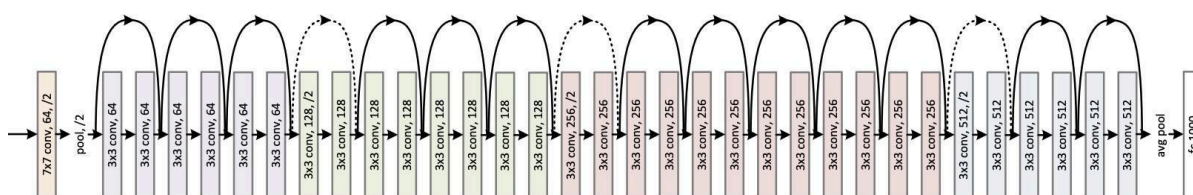


Figura 2.10: Arquitectura de ResNet34. Las líneas negras representan las conexiones de los bloques residuales como en la Figura 2.9. Las líneas negras discontinuas también son conexiones

residuales que aumentan la dimensión de la entrada del bloque. *Extraído de [23].*

2.3.6. SqueezeNet

La red neuronal **SqueezeNet** fue introducida en 2016 con el objetivo de alcanzar con arquitecturas convolucionales más pequeñas, resultados similares obtenidos con arquitecturas mayores, véase el artículo original [24]. Redes neuronales más pequeñas tienen diversas ventajas como facilitar la comunicación en el entramiento distribuido, que las redes neuronales en la nube se actualicen más rápido (como en el caso de los coches autónomos) y que son más factibles de implementar en hardware con memoria limitada. SqueezeNet es capaz de alcanzar la *accuracy* de AlexNet en *ImageNet* con 50 veces menos parámetros y con un tamaño comprimido de 0,5MB. Para lograr menos parámetros se utilizan los módulos *fire*. Un módulo *fire* está formado por: una capa convolucional *squeeze* con s_1 *kernels* 1×1 , que alimenta a una capa *expand* formada por e_1 *kernels* 1×1 y e_3 *kernels* 3×3 . Las diferentes salidas de la capa *expand* de los filtros se concatenan finalmente. Por último, se usa una capa convolucional con *dropout* de 0.5 y una capa de *average pool* como parte clasificadora de la red. Se usa ReLU como función de activación después de cada capa convolucional. Se puede ver la arquitectura de SqueezeNet1.1 en la figura 2.11.

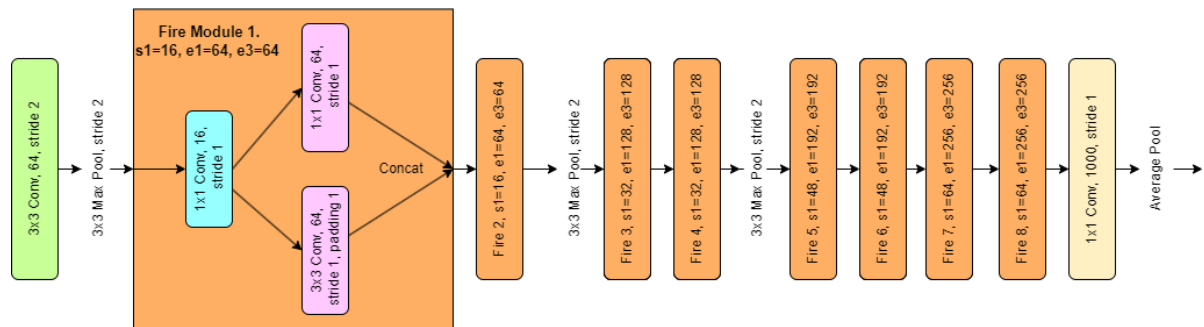


Figura 2.11: Arquitectura de SqueezeNet1.1. En naranja los módulos *fire*, en azul la capa *squeeze*, en rosa la capa *expand* y en amarillo la capa clasificadora convolucional.

2.3.7. MobileNet

La primera versión de **MobileNet** fue presentada en 2017, véase [27]. La principal idea era crear redes convolucionales pequeñas, eficientes y rápidas, principalmente para poder ser implementadas en dispositivos móviles. MobileNet para ello utiliza un nuevo tipo de convolución llamada convolución separable *depthwise*. La idea es reemplazar una capa convolucional estándar por una versión factorizada formada por 2 capas separadas, que nos reducen el número de pesos y operaciones a realizar. Imaginemos que se tiene una imagen RGB (3 canales de entrada) y se quiere aplicar una capa convolucional con 32 filtros de tamaño $3 \times 5 \times 5$ (2400 pesos). La convolución separable *depthwise* estará formada por una primera capa convolucional llamada *depthwise* con tres filtros $1 \times 5 \times 5$ (75 pesos), uno para cada canal de la imagen entrante, manteniendo el mismo número de canales en la salida. La segunda capa convolucional estará

formada por 32 filtros $3 \times 1 \times 1$ (96 pesos). Las dos implemen-

taciones tienen la misma dimensión de salida y canales, pero una usa 2400 pesos y la otra 171 en total. La segunda versión de MobileNet, véase [28], también utiliza bloques residuales como ResNet. La estructura de estos bloques se puede ver en la figura 2.12, donde se utiliza *batch normalization* después de cada capa convolucional y la función de activación $\text{ReLU6} = \min(\text{ReLU}(x), 6)$. Para ver la arquitectura de MobileNetv2 se remite a la tabla 2 de [28].

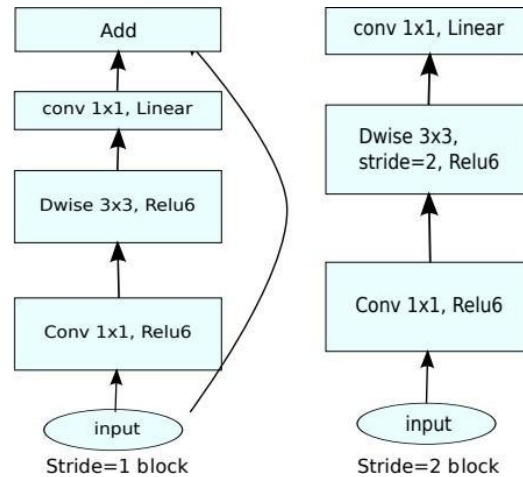


Figura 2.12: Estructura del bloque residual de MobileNetv2. A la izquierda el bloque residual con *stride* 1, a la derecha el bloque sin conexión residual y con *stride* 2. *Extraído de [28].*

2.4. Aprendizaje supervisado y no supervisado

El objetivo principal de las redes neuronales es alcanzar el mejor modelo capaz de resolver un problema, modificando sus parámetros mediante el entrenamiento con distintos datos. Dependiendo de si se encuentran estos datos o no etiquetados se pueden distinguir principalmente el aprendizaje supervisado, semi-supervisado, auto-supervisado y no supervisado, véase [4].

El **aprendizaje supervisado** consiste en el entrenamiento de modelos a partir de datos de entrenamiento etiquetados. Estos datos estarán formados por el dato de entrada y el dato de salida deseado. De esta manera se podrá corregir al modelo y actualizar sus parámetros cuando su salida no coincida con la deseada. Aparte del entrenamiento de redes neuronales con datos etiquetados que ya se ha explicado, hay más métodos que emplean este tipo de aprendizaje los clasificadores kNN, la regresión logística, la clasificación bayesiana, etc.

El **aprendizaje no supervisado** consiste en el entrenamiento de modelos a partir de datos de entrenamiento no etiquetados. Estos datos solo estarán formados por los datos de entrada. Este aprendizaje aprenderá patrones de los datos de entrada sin la necesidad de intervención humana. Se suele usar en problemas de *clustering* para agrupar datos similares juntos y problemas de reducción de dimensionalidad para comprimir datos.

El **aprendizaje semi-supervisado** consiste en el entrenamiento de modelos a partir de unos pocos datos de entrenamiento etiquetados y muchos no etiquetados. Se suele usar en problemas en los que hay muchos datos sin etiquetar y el proceso de etiquetarlos es muy caro. Una aplicación de este aprendizaje es el de obtener etiquetas para los datos no etiquetados a partir de los que sí lo están. Un ejemplo sería entrenar dos modelos y cada uno con un conjunto diferente de datos etiquetados. Después, a partir de los datos no etiquetados se obtienen las salidas esperadas, y se usarían solo las salidas más probables para entrenar a la otra red de manera supervisada con esas nuevas etiquetas. De esta manera se iría aumentando progresivamente y de manera segura el número total de datos etiquetados.

2.5. Aprendizaje auto-supervisado

Para cada tarea diferente el aprendizaje supervisado requiere de recolectar grandes cantidades de datos específicos, limpiarlos y etiquetarlos manualmente. Este proceso suele ser muy costoso, sobre todo si los datos a etiquetar necesitan conocimiento profesional, como puede ser el caso de las imágenes médicas. Por otra parte, el aprendizaje supervisado dista de como los seres humanos aprenden, que suele ser de manera no supervisada y mediante prueba-error.

Por ello se crea el **aprendizaje auto-supervisado**, un tipo de aprendizaje no supervisado que intenta aprender representaciones útiles a partir de datos no etiquetados. Para conseguirlo se crean las llamadas **tareas pretexto**, las cuales aplican una transformación al dato y requieren que el modelo sea capaz de predecir la transformación aplicada. Estas tareas generan etiquetas automáticas de los datos a partir de transformaciones, de manera que serán los propios datos los que supervisarán al sistema, de ahí que se denomine auto-supervisado. Finalmente, el modelo tendrá que ser capaz de entender las características y representaciones de la imagen para poder predecir que transformación ha sido realizada, véase [12]. Las tareas pretexto más conocidas con imágenes son:

- 1.- Rotación [10]: de cada imagen se producen cuatro copias rotándola 0° , 90° , 180° y 270° . De manera que de cada entrada se obtienen 4 imágenes diferentes con sus 4 etiquetas (la rotación aplicada). Por tanto, la tarea pretexto será un problema de clasificación de 4 clases en el que el modelo tendrá que predecir la rotación de la imagen.
- 2.- *Jigsaw* [29]: cada imagen se parte en 9 parches de los cuales se hace una permutación para variar el orden. Esta permutación se le envía a la red y la tarea pretexto es la clasificación de la permutación realizada sobre las 100 posibles clases de permutaciones posibles.
- 3.- Posición relativa del parche [11]: de manera similar a la anterior se divide la imagen

en parches. La tarea pretexto consistirá en que dados 2 parches de la imagen diferentes el modelo sea capaz de clasificar la ubicación relativa de un parche respecto al otro (esta

será la clase) de entre los 8 posibles casos (8 clases en total).

4.- *Colorization* [30]: consiste en transformar imágenes RGB a imágenes en blanco y negro. La tarea del modelo será predecir los colores originales que la formaban.

El objetivo principal del aprendizaje auto-supervisado será que el modelo sea capaz de aprender y extraer representaciones útiles de los datos para poder resolver otras **tareas supervisadas posteriores**. Normalmente, estas tareas posteriores suelen ser tareas como puede la clasificación o la detección de objetos. Para lograr esto se suele realizar **transfer learning**. El *transfer learning* consiste en usar un modelo, el cual ya ha sido entrenado para alguna primera tarea, en una segunda tarea diferente. Para ello a veces se añaden capas a estos modelos, las cuales son entrenadas para realizar la nueva tarea, mientras que los pesos de las capas de la red original no son modificados. Otra manera es modificar los pesos del modelo original para adecuarlo a la nueva tarea, ya que la inicialización de estos es adecuada al haber aprendido buenas representaciones. A este último método se le suele denominar *fine-tuning*. En particular, la finalidad es mejorar el aprendizaje en la tarea supervisada posterior, gracias a las representaciones y el conocimiento aprendido por la red en la tarea pretexto.

Otra manera de realizar aprendizaje auto-supervisado es a través de los llamados **aprendizaje contrastivo** y **no contrastivo**. Ambos usan redes neuronales siamesas, que son redes neuronales que comparten la arquitectura y los mismos pesos, de manera que producen salidas iguales si se les pasa la misma entrada. Esto hace que estas redes sean muy apropiadas para comparar datos. Además, los dos tipos de aprendizaje generarán pares de modificaciones de las imágenes con las que alimentarán a las redes siamesas. En el caso del aprendizaje contrastivo, se alimentará a la red con pares positivos (si las dos modificaciones son de la misma imagen) intentando maximizar la función similitud entre ambas y con pares negativos (si son modificaciones de diferentes imágenes) de los cuales se intentará distanciar o repeler la función de similitud. Un método de aprendizaje auto-supervisado contrastivo es *SimCLR*, véase [31]. En cambio, el aprendizaje no contrastivo solo utilizará pares positivos de modificaciones de una misma imagen intentando maximizar o aumentar la función de similitud entre las modificaciones. Un método de aprendizaje auto-supervisado no contrastivo es **SimSiam**, el cual se detalla en la subsección 2.5.1.

2.5.1. *SimSiam*

El aprendizaje mediante redes siamesas simples, también conocido como **SimSiam**, es un modelo de aprendizaje auto-supervisado no contrastivo introducido en 2020 por *Facebook AI Research*. En esta subsección se explica *SimSiam* y su arquitectura tal y como han sido definidas en el artículo original, véase [1]. *SimSiam* usa redes neuronales siamesas y pares positivos de

modificaciones de imágenes con el objetivo de aprender representaciones útiles. Concretamente, SimSiam aprenderá representaciones que serán invariantes a las transformaciones que se les aplique a las imágenes.

La arquitectura de SimSiam recibirá como entradas dos modificaciones x_1 y x_2 de una misma imagen x . Las dos modificaciones serán procesadas por una red neural llamada *encoder* formada por un *backbone* seguido de un proyector MLP. Se denota al *encoder* como f . Esta red f será la red neuronal siamesa que compartirá los pesos entre las dos modificaciones. Después una cabeza MLP llamada *predictor* que se denotará como h , transformará el output de f de una sola de las modificaciones. A la salida del predictor se denota como $p_1 \triangleq h(f(x_1))$ y a la salida del *encoder* como $z_2 \triangleq f(x_2)$. El objetivo del modelo será minimizar la función similitud del coseno negativo entre p_1 y z_2 :

$$D(p_1, z_2) = - \frac{p_1 \cdot z_2}{\|p_1\|_2 \|z_2\|_2}, \quad (2.12)$$

donde $D(p_1, z_2)$ es el coseno negativo del ángulo entre los vectores p_1 y z_2 debido a la definición del producto escalar usual que se denota como $p_1 \cdot z_2$. En la función además se normalizan los vectores dividiendo los vectores por sus normas l^2 . De manera que, si dos vectores son muy similares, el ángulo entre ellos será muy pequeño y cercano a cero y la función nos devolverá valores cercanos al menos coseno de cero que es -1 , el valor mínimo que puede alcanzar. Se toma el valor negativo del coseno para que en vez de maximizar la función se tenga que minimizar, pudiendo aplicar el descenso del gradiente. A partir de esta función se define la función coste para la red tal que:

$$L = \frac{1}{2} D(p_1, z_2) + \frac{1}{2} D(p_2, z_1). \quad (2.13)$$

Esta función es definida para cada imagen, y la función coste total será la media sobre todas las imágenes del lote. Un importante componente del método es parar el descenso del gradiente de z_1 y z_2 , que se denota como *stopgrad*. Esto hace que la función de la ecuación 2.13 se convierta finalmente en la función coste final:

$$L = \frac{1}{2} D(p_1, \text{stopgrad}(z_2)) + \frac{1}{2} D(p_2, \text{stopgrad}(z_1)). \quad (2.14)$$

De manera que el *encoder* f en x_2 no recibe gradiente de z_2 en el primer término, pero si recibe gradiente de p_2 en el segundo término. De manera similar para x_1 . La arquitectura general de *SimSiam* se muestra en la figura 2.13.

La arquitectura del *backbone*, del proyector MLP y del predictor están formadas por:

- 1.- El *backbone* o extractor de características es una red formada por las capas extractoras de características (las capas convoluciones y de *pooling*) de una red neuronal. En el artículo original se utilizan las capas extractoras de características de ResNet50. Es el primer componente del *encoder*.

2.- El proyector MLP estará formado por un MLP con 3 capas FC. La primera capa recibe como entradas las salidas del *backbone* (el número de entradas dependerá de la red usada como *backbone*) y tendrá 2048 neuronas. La segunda capa también tendrá 2048 neuronas. La última capa tendrá d neuronas y será la dimensión final, se suele inicializar $d = 2048$,

aunque se puede reducir si es necesario. A las salidas de todas las capas del proyector se les aplica *batch normalization*. Se usa como función de activación ReLU, menos en la salida que no se usa ninguna función. Es el segundo componente del *encoder*.

3.- El predictor MLP está formado por 2 capas. La primera capa tendrá d entradas que son las salidas del proyector MLP y estará formada por $\lfloor d/4 \rfloor$ neuronas, en nuestro caso 512. La salida de la última capa tendrá dimensión de tamaño d , es decir estará formada por d

neuronas. Solo se aplica *batch normalization* y ReLU en la primera capa.

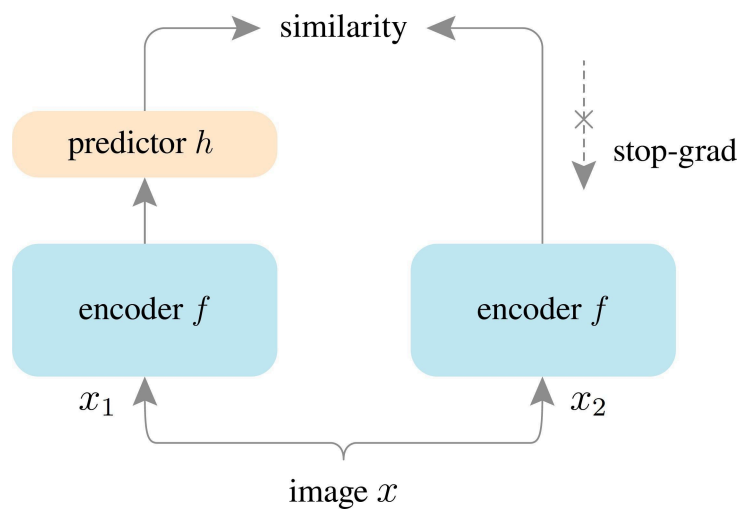


Figura 2.13: Arquitectura de *SimSiam*. Las modificaciones pasan por el encoder f con los mismos pesos. A una de las salidas del encoder se le aplica el predictor h y se intenta minimizar la función de similitud 2.14 a la vez que se para el gradiente en el lado derecho. *Extraído de [1].*

Uno de los problemas del aprendizaje no contrastivo es que los modelos pueden colapsar a una función constante trivial, ya que a salida constante las salidas son siempre iguales y se minimizará la función de coste siendo siempre -1 . El aprendizaje contrastivo lo soluciona añadiendo pares negativos, ya que en este caso entradas de imágenes diferentes no pueden tener la misma salida constante ya que quieres maximizar la distancia entre estas salidas. Para solucionar este problema *SimSiam* introduce la operación de stopgrad, la cual se demuestra en el artículo original que es responsable de que el modelo no colapse en soluciones constantes. Además, también se muestra de manera empírica que es necesario entrenar los pesos del predictor h para alcanzar una buena representación, ya que sin él o con una distribución fija de los pesos los modelos no convergen y el error es alto. Esto es debido a que el predictor debe poder ser capaz de adaptarse a las diferentes representaciones. Además a diferencia del aprendizaje contrastivo, *SimSiam* no necesita de un gran *batch size* para obtener buenos resultados, haciéndolo más idóneo en casos en los que no se disponga de mucha memoria.

3.1. Entorno de desarrollo

Este trabajo ha sido realizado en el lenguaje de programación Python en un entorno Conda usando *Jupyter notebooks*. Concretamente se ha utilizado el framework de Python llamado **Pytorch** desarrollado por *Meta AI* para la creación de proyectos de aprendizaje automático sobre redes neuronales, véase [32]. Esta librería nos permite crear conjuntos de datos, crear redes neuronales, optimizar sus pesos mediante el entrenamiento y finalmente poder guardarlas. Esto es debido a que *Pytorch* utiliza tensores. Un tensor es una estructura de datos similar a los arrays que permite ser ejecutada y guardada en la GPU. Con estos tensores se representarán las entradas, salidas y parámetros de las redes. Además, estos tensores están optimizados para la diferenciación automática, la cual es un paso esencial en el descenso del gradiente y la *backpropagation* para el entrenamiento de las redes.

Normalmente en el entrenamiento de las redes estamos interesados en realizar las operaciones de manera paralela en la GPU y no en la CPU. Esto es debido a que las GPUs tienen muchos más núcleos aunque de menor velocidad que las CPUs, lo que las hacen más idóneas para realizar las tareas en paralelo y disminuir el tiempo de entrenamiento. Para poder realizar cálculos de manera paralela en una GPU de NVIDIA, *Pytorch* usa el soporte de CUDA. CUDA es una plataforma de computación paralela y un modelo de programación desarrollados por NVIDIA.

Para guardar los parámetros que se miden durante el entrenamiento de las redes se ha usado la herramienta **Neptune AI**¹. Por último, se usa el paquete **torchvision** de *Pytorch* el cual consiste en un conjunto de bases de datos de imágenes populares, arquitecturas de redes neuronales convolucionales para *ImageNet* y transformaciones de imágenes para la visión por computador.

En nuestro caso se ha utilizado el sistema operativo Windows 10, una CPU AMD Ryzen 5 3600X 3.8GHz, 16GB de RAM y una GPU NVIDIA GeForce RTX 2060 Super 8GB GDDR6. Para la instalación de CUDA se descargó de la página web de NVIDIA el instalador para la versión 11.8². Posteriormente, se realizó la instalación de *Pytorch* 2.0 y de *Neptune* utilizando Conda (ver código A.8 y A.9).

¹ *Neptune AI*, <https://neptune.ai>

² NVIDIA CUDA 11.8, <https://developer.nvidia.com/cuda-11-8-0-download-archive>

3.2. Imagenette e ImageWoof

Imagenette³ es un subconjunto de 10 clases fácilmente clasificables de *ImageNet*: tenca, springer inglés, reproductor de casetes, motosierra, iglesia, trompa, camión de la basura, surtidor de gasolina, pelota de golf y paracaídas. El orden anterior será la etiqueta de las clases, se empieza etiquetando con 0 a la primera (la tenca) hasta llegar a la etiqueta 9 con la décima (el paracaídas). En la figura 3.1 se puede ver un ejemplo de cada clase.



Figura 3.1: Ejemplos de cada clase de *Imagenette*.

Imagenette está formada por un total de 13394 imágenes siendo esta dividida en 9469 imágenes de entrenamiento (70,69 %) y 3925 para la validación (29,31 %). El número de imágenes para cada clase en los conjuntos de entrenamiento y validación se puede ver en la tabla 3.1. La cantidad de imágenes es pareja para cada clase, a excepción de la clase con etiqueta 3 (motosierra) que es la que menos tiene, con alrededor de 100 imágenes menos de entrenamiento.

Etiqueta	0	1	2	3	4	5	6	7	8	9
Entrenamiento	963	955	993	858	941	956	961	931	951	960
Validación	387	395	357	386	409	394	389	419	399	390

Tabla 3.1: Número de imágenes por clase en *Imagenette* para el entrenamiento y validación.

ImageWoof es un subconjunto de 10 clases de *ImageNet* de diferentes razas de perros: Shih-Tzu, crestado rodesiano, beagle, foxhound inglés, border terrier, terrier australiano, golden retriever, antiguo pastor inglés, samoyedo y dingo. Es de los mismos creadores que *Imagenette* y se puede encontrar en el mismo repositorio. Debido a que la clase es la raza del perro, la similitud entre imágenes será mucho mayor que con *Imagenette* y por tanto más complicada será su clasificación. El orden anterior será la etiqueta de las clases, se empieza etiquetando con 0 a la primera (Shih-Tzu) y se termina etiquetando con 9 a la décima (dingo). En la figura 3.2 se puede ver un ejemplo de cada clase.



Figura 3.2: Ejemplos de cada clase de *ImageWoof*.

ImageWoof está formada por un total de 12954 imágenes siendo este dividida en 9025 imágenes de entrenamiento (69,67 %) y 3929 para la validación (30,33 %). El número de imágenes para cada clase en los conjuntos de entrenamiento y validación se puede ver en la tabla 3.2. Para la clase con etiqueta 3 (foxhound inglés) hay muchos menos imágenes, alrededor de 350, para el entrenamiento. Esto afectará negativamente a la *accuracy* de esta clase y a la general de los modelos.

Etiqueta	0	1	2	3	4	5	6	7	8	9
Entrenamiento	941	942	932	580	949	943	949	928	921	940
Validación	409	408	418	224	401	407	401	422	429	410

Tabla 3.2: Número de imágenes por clase en *ImageWoof* para el entrenamiento y validación.

3.3. Implementación de la clasificación supervisada

El objetivo será entrenar las redes convolucionales *Resnet34*, *VGG13*, *AlexNet*, *SqueezeNet* y *MobileNetV2* de manera supervisada para la clasificación de imágenes. Primero se entrenan para *Imagenette* y posteriormente para *ImageWoof*. De las redes neuronales nombradas se usarán los modelos implementados en la librería *torchvision* de *Pytorch*. Estos modelos están implementados para la clasificación de *ImageNet* con 1000 clases. En nuestro caso se modificará la última capa de cada red para solo clasificar 10 clases.

En el ajuste de los hiperparámetros de las distintas redes se han usado los hiperparámetros recomendados de los artículos originales para la clasificación de *ImageNet*, a excepción del *batch size* debido a las limitaciones de hardware y de memoria al usar tamaños muy grandes. Todos los modelos han sido entrenados 300 épocas cada uno con un *batch size* de 64. Además, se ha usado el optimizador SGD y la función de coste *Cross-Entropy Loss* (ecuación 2.8) tras aplicar *softmax* a las salidas (ecuación 2.4). El resto de hiperparámetros específicos de cada red son:

- *ResNet34* [23]: se ha utilizado un *learning rate* de 0,1 y se ha reducido por un factor de 0,1 cada vez que la *loss* de la validación se ha aplanado. Se ha usado *weight decay* de 0,0001 y *momentum* de 0,9.
- *VGG13* [22] y *AlexNet* [13]: se ha utilizado un *learning rate* de 0,01 y se ha reducido por un factor de 0,1 cada vez que la *loss* de la validación se ha aplanado. Se ha usado *weight decay* de 0,0005 y *momentum* de 0,9.
- *SqueezeNet* [24]: se ha utilizado un *learning rate* de 0,004 y se ha ido reduciendo de manera lineal. Se ha usado *weight decay* de 0,0002 y *momentum* de 0,9.
- *MobileNetV2* [28]: se ha utilizado un *learning rate* de 0,045 y se ha reducido por un factor de 0,98 después de cada época. Se ha usado *weight decay* de 0,00004 y *momentum* de 0,9.

Se tomarán como métricas al final de cada época la **loss** y la **accuracy top-1 y top-5** del entrenamiento y de la validación. La *accuracy top-1*, también conocida simplemente como *accuracy*, es el número de predicciones correctas entre el total de predicciones realizadas. En la *accuracy top-5* la predicción es correcta si la etiqueta de la imagen se encuentra entre las 5 primeras salidas de la red con mayor probabilidad. Finalmente se medirá el **tiempo total** que se tardó en entrenar cada red.

Para reducir el *overfitting* se ha usado *data augmentation* de las imágenes de entrenamiento, concretamente se han utilizado las transformaciones descritas en [23]. Primero se realiza un recorte aleatorio de la imagen y se reescala a un tamaño de 224×224 píxeles, el tamaño de las entradas de los modelos. A este recorte se le aplica un volteo horizontal con una probabilidad $p = 0,5$. Posteriormente, se cambia el brillo, contraste y saturación de la imagen. Finalmente, se convierte la imagen en un tensor y se normaliza con la media y desviación estándar de *ImageNet* (véase el código A.4). En el caso de las imágenes de validación simplemente se reajustan los tamaños de los datos ya que se tienen imágenes de diversos tamaños y se necesitan que tengan tamaño 224×224 (véase el código A.5). Para ello simplemente se reescalan a un tamaño de 256×256 y posteriormente se realiza un recorte central de tamaño 224×224 . Por último, se transforman en tensores y se normalizan para poder ser tratados por los modelos.

3.4. Implementación de *SimSiam*

El objetivo es realizar el preentrenamiento de las capas extractoras de características de las redes neuronales convolucionales *Resnet34*, *VGG13*, *AlexNet*, *SqueezeNet* y *MobileNetV2* mediante *SimSiam*, utilizando los conjuntos de datos *Imagenette* y posteriormente *ImageWoof*. Para ello se implementarán estos extractores de características como *backbone* del *encoder* de *SimSiam* (ver figura 2.13).

Para el preentrenamiento se usará una dimensión final $d = 2048$ tanto en el proyector y predictor MLP (véase la Sección 2.5.1). El número de entradas del proyector MLP de *SimSiam* tendrá un tamaño diferente dependiendo de la dimensión del vector de características de la salida de los extractores de características (véase el código implementado en la Sección A.1). En concreto para

3.3. IMPLEMENTACIÓN DE LA CLASIFICACIÓN SUPERVISADA

cada una de las redes se tiene que:

- *AlexNet* : el extractor de características está formado por todas las capas de la red, exceptuando las 3 últimas capas clasificadoras MLP (ver figura 2.7). El vector de características es el devuelto por la última capa *max pool* con una dimensión de **9216**.
- *VGG13*: el extractor de características está formado por todas las capas de la red, exceptuando las 3 últimas capas clasificadoras MLP (ver figura 2.8). El vector de características es el devuelto por la última capa *max pool* con una dimensión de **25088**.
- *ResNet34*: el extractor de características está formado por todas las capas de la red, exceptuando la última capa clasificadora MLP (ver figura 2.10). El vector de características es el devuelto por la capa de *average pool* con una dimensión de **512**.
- *SqueezeNet* : el extractor de características está formado por todas las capas de la red, exceptuando la última capa convolucional que actúa como clasificadora (ver figura 2.11). El vector de características es el devuelto por la última capa convolucional con una dimensión de **512x13x13** siempre que la entrada sea una imagen de tamaño 224×224 .
- *MobileNetV2*: el extractor de características está formado por todas las capas de la red, exceptuando la última capa clasificadora MLP. El vector de características es el devuelto por la última capa convolucional con una dimensión de **1280**.

Para el preentrenamiento se usa los hiperparámetros del artículo original [1]. Concretamente se usa SGD como optimizador y como función de coste la ecuación 2.14. Un *learning rate* de $lr \times \frac{BatchSize}{256}$ con un $lr = 0,05$ base. El *learning rate* se decae de manera cosenoidal a lo largo del entrenamiento, pero manteniéndose fijo en el predictor h . El *weight decay* es de 0,0001 y *momentum* de 0,9. Como se muestra en el artículo, *SimSiam* obtiene buenos resultados con *batch sizes* de 64 a 4096 (potencias de dos). Obteniendo los mejores resultados con los tamaños de 256 a 2048. Debido a las limitaciones de hardware se usa como *batch size* la mayor potencia de dos posible con nuestra memoria que es 64, menos en VGG13 que al tener más parámetros y mayor dimensión de los datos se usa de 32.

Se realizará el preentrenamiento durante 800 épocas y se guardarán los correspondientes modelos a las 100, 200, 400 y 800 épocas de preentrenamiento. Cada 5 épocas se usará un clasificador kNN con $k = 1$ como monitor del progreso del preentrenamiento. Este kNN calculará la salida del *encoder* de *SimSiam* con cada imagen del conjunto de validación. Una vez obtenidas todas las salidas del *encoder* de todas las imágenes de validación, el kNN para cada salida calculará su similitud coseno (ver ecuación 2.12) con todas las demás salidas y asignará como predicción a cada imagen la etiqueta de la imagen con la salida con mayor similitud. Se medirá la **accuracy del kNN** y la **loss** durante el preentrenamiento, así como el **tiempo** total necesario para preentrenar cada modelo. Al final del preentrenamiento, se muestra una **proyección 2D con t-SNE** de las salidas de los extractores de características preentrenados con *SimSiam*, al ser alimentados con las imágenes de validación. El algoritmo t-SNE [33] es un método para poder visualizar datos de alta dimensionalidad como puntos en un plano 2D o 3D. De manera que se podrán visualizar las distancias entre las distintas salidas.

Respecto al *data augmentation* de los datos se realiza el propuesto en [1]. Primero se realiza un

Desarrollo y Diseño

recorte de tamaño aleatorio de la imagen y se reescala a un tamaño de 224×224 (el tamaño de entrada de las redes convolucionales que se usan). Después se cambia el brillo, contraste y saturación de la

imagen con una probabilidad $p_1 = 0,8$ y se transforma la imagen a blanco y negro con una probabilidad $p_2 = 0,2$. A continuación, se le añade ruido gaussiano a la imagen con probabilidad $p_3 = 0,5$ y se voltea la imagen horizontalmente con probabilidad $p_4 = 0,5$. Por último, se transforma la imagen en un tensor y se normaliza con la media y desviación de *ImageNet* (véase código A.6). De esta manera se obtendrán las dos modificaciones de una misma imagen, que serán las entradas de *SimSiam*.

3.5. Implementación de la clasificación auto-supervisada

Dados los extractores de características preentrenados mediante *SimSiam* con *Imagenette* e *ImageWoof* de la Sección 3.4, nuestro objetivo será entrenar capas clasificadoras de manera supervisada sobre las **salidas congeladas** de estos extractores de características. A cada extractor de características se le añadirán las capas clasificadoras correspondientes a su arquitectura original. De esta manera, la arquitectura de los modelos supervisados y auto-supervisados será la misma, con la principal diferencia de que los pesos de las capas extractoras de características en el entrenamiento no se modificarán en los auto-supervisados. Es decir, en el supervisado se entrenan todos los parámetros de los modelos, mientras que en el auto-supervisado solo se entrenarán los parámetros de las capas clasificadoras mientras se mantienen fijos (congelados) los de las capas extractoras de características.

La idea principal es que se realiza *transfer learning*, al usar un modelo que ha sido preentrenado de manera auto-supervisada en una segunda tarea supervisada (clasificar las imágenes de *Imagenette* e *ImageWoof*). Debido a que los clasificadores MLP de pocas capas tienen poca capacidad de discriminación y de clasificar de manera precisa imágenes, su rendimiento dependerá de las representaciones útiles aprendidas por el extractor de características auto-supervisado.

Se utilizarán los extractores de características de los modelos preentrenados 100, 200, 400 y 800 épocas de manera auto-supervisada, tanto en *Imagenette* como en *ImageWoof*. Para el entrenamiento de las capas clasificadoras se ha usado SGD y la función de coste *Cross-Entropy loss* tras aplicar *softmax* a las salidas. Se ha utilizado un *batch size* de 256 en todos los modelos, excepto en VGG13 que al tener más parámetros se ha usado uno de 128 por limitaciones de hardware. En *ResNet34*, *AlexNet* y *MobileNetV2* se ha usado un *learning rate* de 0,4, en VGG13 de 0,2 y en *SqueezeNet* de 30. En todos los modelos el *learning rate* se decae de manera cosenoidal y se utiliza un *momentum* de 0,9 y *weight decay* de 0. Los modelos han sido entrenados durante 90 épocas. De esta clasificación auto-supervisada también se tomarán las métricas correspondientes a la **accuracy top-1**, **accuracy top-5** y **loss** del entrenamiento y validación. Con el objetivo de comparar estas métricas con las obtenidas durante la clasificación supervisada. Respecto al *data*

Desarrollo y Diseño

augmentation de los datos de entrenamiento, a cada imagen se le realiza un recorte aleatorio y se reescala a tamaño 224×224 , se voltea horizontalmente la imagen con probabilidad $p = 0,5$, se convierte a tensor y se normaliza (véase código A.7). Respecto a los datos de validación se realizan las mismas transformaciones que en el caso supervisado (véase código A.5).

4.1. Resultados de la clasificación supervisada

En la tabla 4.1 se muestran los resultados obtenidos para la *accuracy* top-1 y top-5 en la clasificación supervisada con la implementación especificada en la Sección 3.3 para las redes neuronales *ResNet34*, *VGG13*, *AlexNet*, *SqueezeNet1.1* y *MobileNetV2*.

Red	<i>Imagenette</i>		<i>ImageWoof</i>	
	Top-1 Acc.	Top-5 Acc.	Top-1 Acc.	Top-5 Acc.
<i>ResNet34</i>	91	99, 44	85, 08	98, 34
<i>VGG13</i>	<u>91, 84</u>	<u>99, 46</u>	<u>85, 28</u>	<u>98, 57</u>
<i>AlexNet</i>	88, 81	99, 13	78, 41	97, 68
<i>SqueezeNet1.1</i>	87, 79	99	80, 52	98, 32
<i>MobileNetV2</i>	90, 39	99, 28	82, 92	98, 44

Tabla 4.1: *Accuracy* top-1 y top-5 (%) de cada arquitectura en la clasificación supervisada de *Imagenette* e *ImageWoof*. Se subraya el mejor resultado en cada caso.

Se puede apreciar que la arquitectura que obtiene los mejores resultados es *VGG13*, seguida de *ResNet34* y *MobileNetV2*. *AlexNet* obtiene los peores resultados para la clasificación de *ImageWoof*, mientras que *SqueezeNet1.1* obtiene los peores resultados para la clasificación de *Imagenette*. Esto se debe a que *VGG13* y *ResNet34* son las arquitecturas con mayor número de capas y su diseño se centra en mejorar la *accuracy* en la clasificación de imágenes, mientras que *AlexNet* es la arquitectura más anticuada y con menor número de capas, y *SqueezeNet* y *MobileNetV2* son arquitecturas más ligeras para dispositivos móviles.

El tiempo de entrenamiento de cada red para la clasificación supervisada de *Imagenette* e *ImageWoof* se muestra en la figura 4.1. Siempre se tarda menos en el caso de *ImageWoof* debido a que este conjunto de datos tiene menos imágenes de entrenamiento que *Imagenette*. *VGG13* es la arquitectura que más tarda en ser entrenada debido al tamaño de las entradas de sus capas y a que tiene un total de 129 millones de parámetros (en el caso de 10 clases), frente a los 21, 28 millones de *ResNet34*, 57 millones de *AlexNet*, 2, 23 millones de *MobileNetV2* y 0, 727 millones de *SqueezeNet1.1*.



Figura 4.1: Tiempos de entrenamiento para la clasificación supervisada en minutos. Se muestra los casos con *Imagenette* e *ImageWoof*.

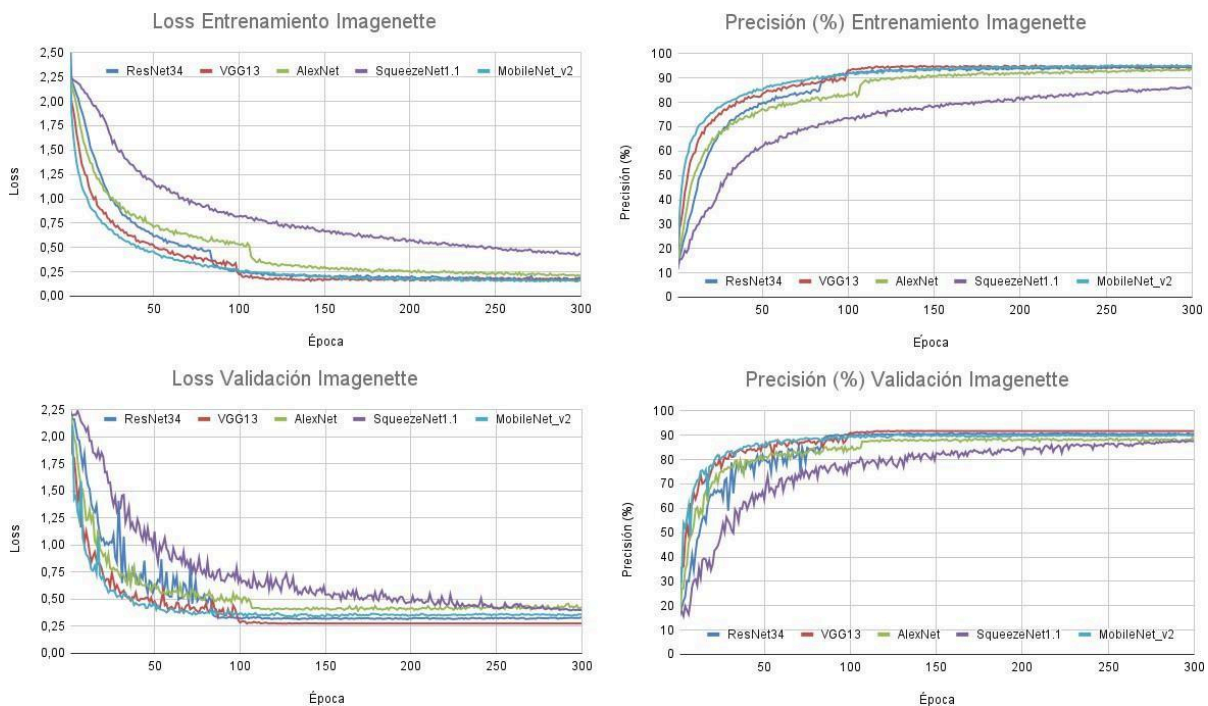


Figura 4.2: Gráficas de la *loss* y la *accuracy* top-1 del entrenamiento y la validación en la clasificación supervisada de *Imagenette*.

En la figura 4.2 se pueden observar las gráficas que muestran la evolución de la *accuracy* top-1 y la *loss* del entrenamiento y la validación para *Imagenette*. Análogamente para el caso de *ImageWoof* en la figura 4.3. Todas las arquitecturas acaban convergiendo y se evita el *overfitting* al no obtener una *accuracy* en el entrenamiento muy alta a costa de la de validación. En el caso de *ImageWoof* se obtienen peores resultados en la *loss* y la *accuracy* ya que *ImageWoof* tiene menos imágenes de entrenamiento y a que estas imágenes son más similares entre ellas al ser todas perros. Por último, se puede observar que *SqueezeNet1.1* es la arquitectura que más épocas necesita para converger tanto en la *loss* como en la *accuracy* al ser la arquitectura con menor número de parámetros.

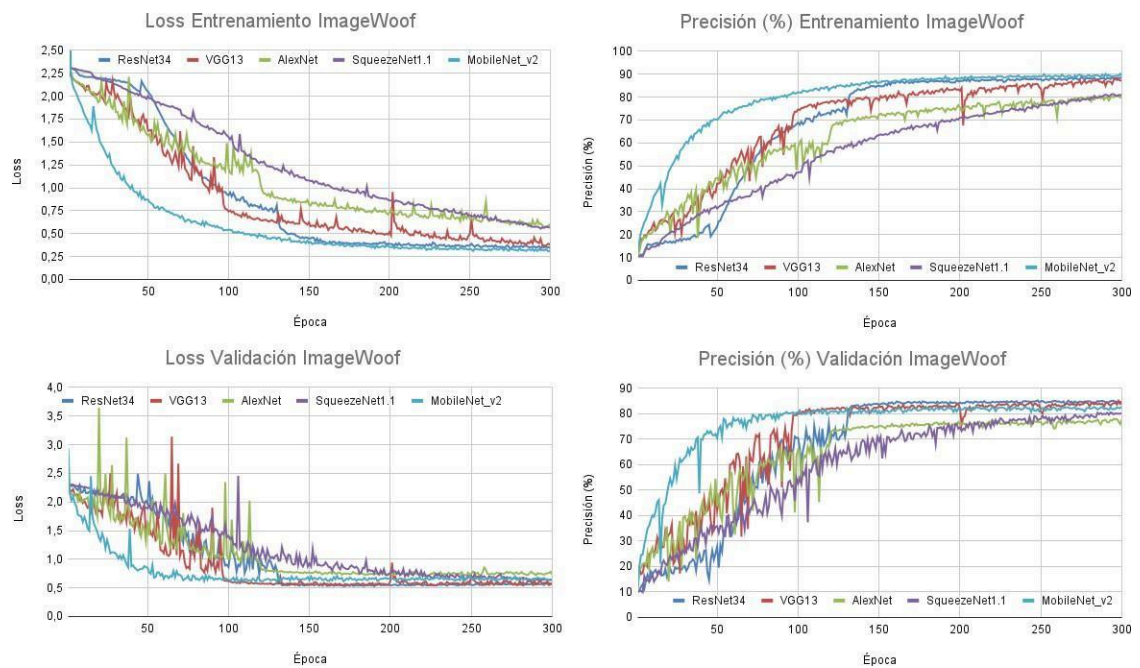


Figura 4.3: Gráficas de la *loss* y la *accuracy* top-1 del entrenamiento y la validación en la clasificación supervisada de *ImageWoof*.

4.2. Resultados del preentrenamiento con *SimSiam*

Los resultados que se presentan del preentrenamiento con *SimSiam* se han obtenido siguiendo la implementación propuesta en la Sección 3.4. Los extractores de características han sido preentrenados con *SimSiam* durante 800 épocas y el tiempo de preentrenamiento de cada red se muestra en la figura

4.4. Al igual que en el caso supervisado, VGG13 es el modelo que más tiempo necesitó debido al tamaño de las entradas de sus capas y al número de parámetros. Durante el preentrenamiento se ha medido la *loss* (ver ecuación 2.14) y se ha realizado una clasificación supervisada kNN con $k = 1$ como monitor para comprobar el progreso. En la figura 4.5 se pueden observar las gráficas que muestran la evolución de la *loss* y la *accuracy* del kNN durante las 800 épocas de preentrenamiento

con *SimSiam* para *Imagenette*. Análogamente en la figura 4.6 para el caso de *ImageWoof*.

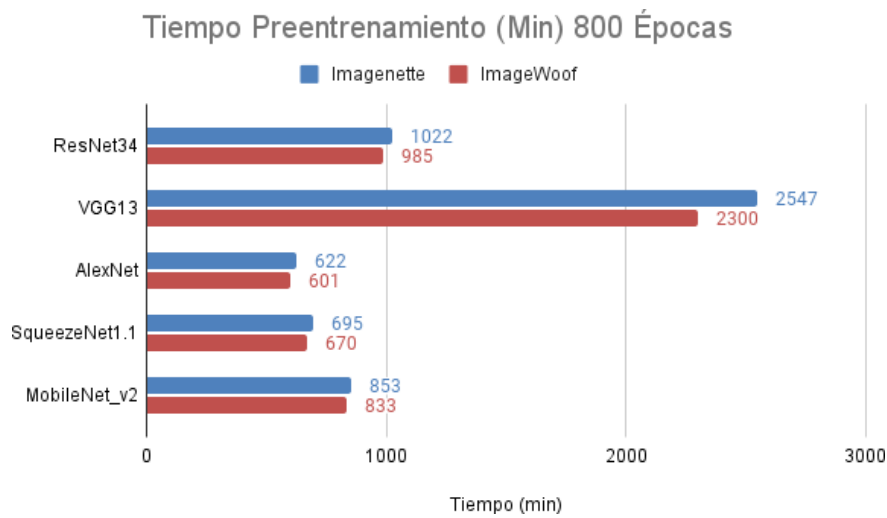


Figura 4.4: Tiempos en minutos de los preentrenamientos con *SimSiam* para *Imagenette* e *ImageWoof*.

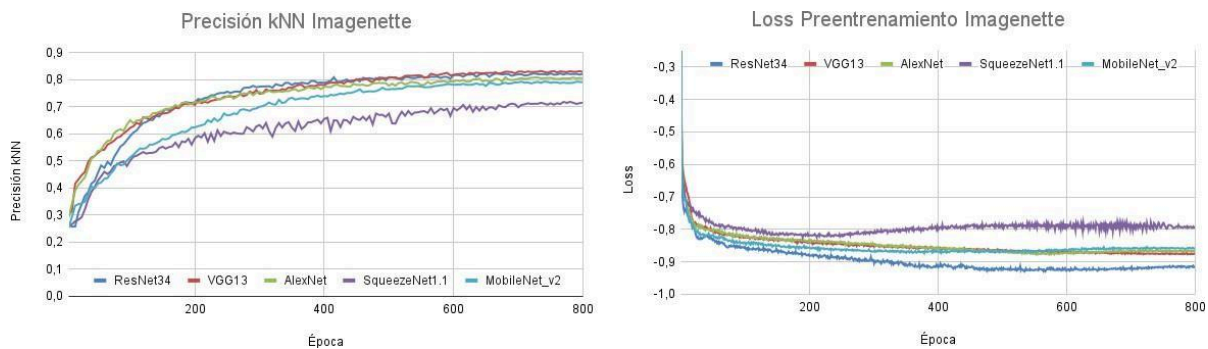


Figura 4.5: Gráficas de la *loss* y la *accuracy* top-1 del clasificador kNN durante el preentrenamiento con *SimSiam* usando la base de datos *Imagenette*.

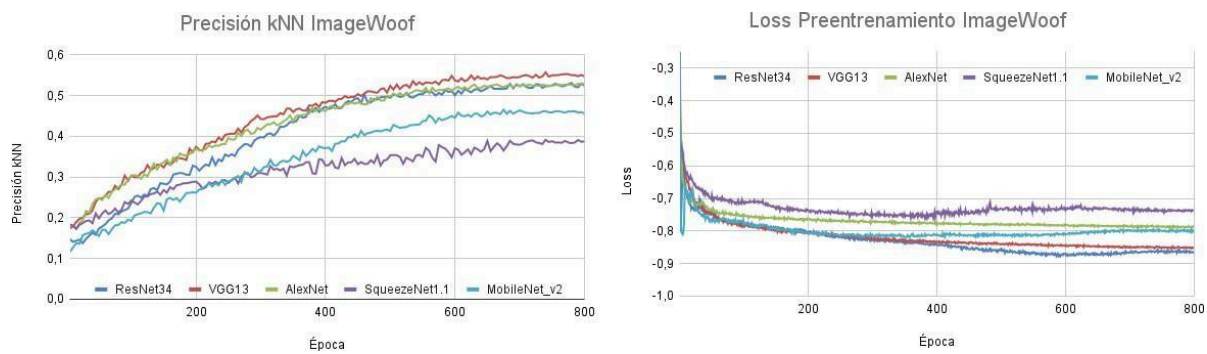


Figura 4.6: Gráficas de la *loss* y la *accuracy* top-1 del clasificador kNN durante el preentrenamiento con *SimSiam* usando la base de datos *ImageWoof*.

Red	Accuracy Imagenette	Accuracy ImageWoof
ResNet34	82, 40	53, 09
VGG13	<u>83, 27</u>	<u>55, 71</u>
AlexNet	80, 94	52, 92
SqueezeNet1.1	71, 87	39, 16
MobileNetV2	79, 32	46, 54

Tabla 4.2: Mejor *accuracy* (%) del clasificador kNN obtenida durante el preentrenamiento con *SimSiam* para *Imagenette* e *ImageWoof*.

El número de parámetros en el preentrenamiento con *SimSiam* es diferente al caso supervisado, ya que solo preentrenamos los extractores de características. En concreto, VGG13 tiene 9, 4 millones de parámetros en su extractor de características, *ResNet34* 21, 2 millones, *AlexNet* 2, 46 millones, *MobileNetV2* 2, 2 millones y *SqueezeNet1.1* 0, 72 millones. En la tabla 4.2 se muestran las mejor *accuracy* obtenida del clasificador kNN para cada arquitectura durante el preentrenamiento con *SimSiam*. Obteniendo VGG13 la mejor *accuracy*, seguida de *ResNet34*, *AlexNet*, *MobileNetV2* y *SqueezeNet1.1*. Se puede observar que a mayor número de parámetros en los extractores de características mejor *accuracy* se obtiene en el clasificador kNN, a excepción de *ResNet34* y VGG13. Esto se puede deber a que *ResNet34* al principio reduce mucho la dimensión de las entradas, debido a una primera convolución con *stride* 2 seguida de una capa *max pool* (ver figura 2.10). En cambio, VGG13 tarda más en reducir la dimensión de las entradas (ver figura 2.8) y trabaja con entradas más grandes durante más capas que *ResNet34*, lo cual le puede permitir obtener mejores representaciones.

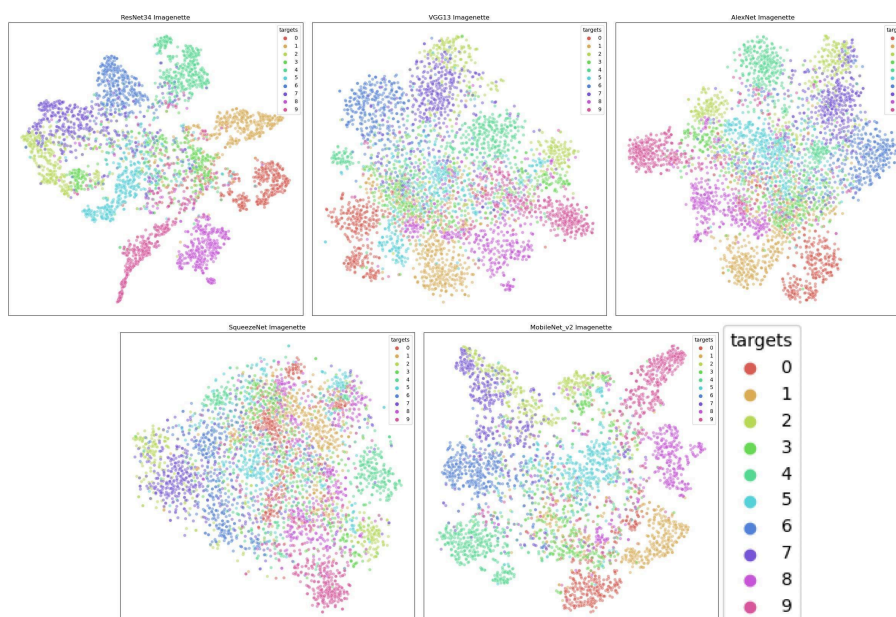


Figura 4.7: Visualización de los vectores de características en 2D de las imágenes de validación de *Imagenette* utilizando t-SNE. De izquierda a derecha se tienen los *backbones*: *ResNet34*, VGG13, *AlexNet*, *SqueezeNet1.1* y *MobileNetV2*. Cada color es una clase diferente.

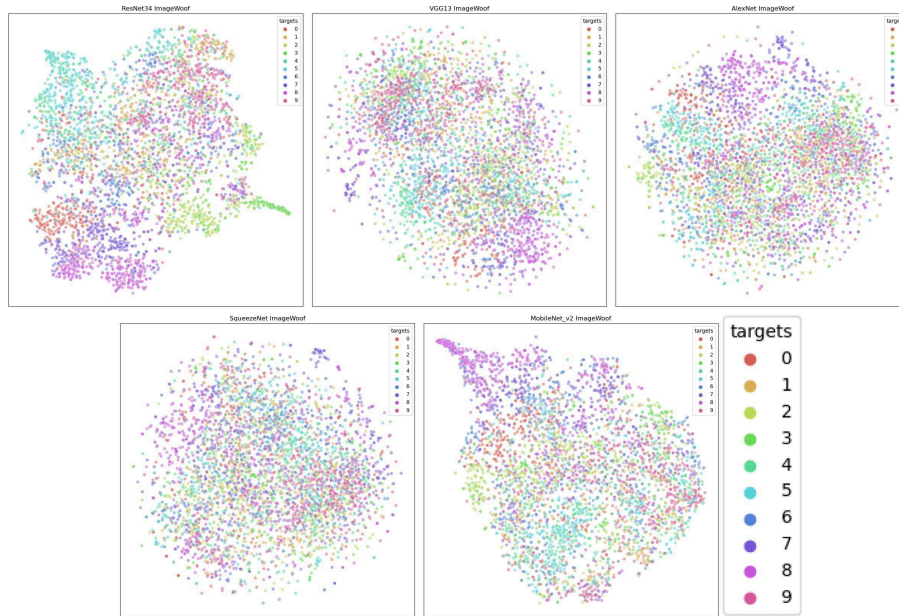


Figura 4.8: Visualización de los vectores de características en 2D de las imágenes de validación de *ImageWoof* utilizando t-SNE. De izquierda a derecha se tienen los *backbones*: *ResNet34*, *VGG13*, *AlexNet*, *SqueezeNet1.1* y *MobileNetV2*. Cada color es una clase diferente.

En la figura 4.7 se pueden ver los vectores de características de las imágenes de validación de *Imagenette* en 2D. Estos vectores de características son las salidas que producen los extractores de características, tras ser preentrenados con *SimSiam*, con las imágenes de validación como entradas. Se ha utilizado el algoritmo t-SNE para reducir la dimensión de las salidas a 2D y se representa cada salida como un punto, siendo el color la clase. En la figura 4.8 se muestran las salidas para *ImageWoof* en 2D. Se puede apreciar que las salidas están mucho más agrupadas y son más fácilmente separables en el caso de *Imagenette* que en el de *ImageWoof*, donde están mucho más dispersas. De esta manera se puede visualizar las distancias entre las salidas y el resultado que nos proporciona kNN, siendo mejor la *accuracy* para *Imagenette* que para *ImageWoof*.

4.3. Resultados de la clasificación auto-supervisada

Para cada arquitectura se han guardado los extractores de características preentrenados durante 100, 200, 400 y 800 épocas con *SimSiam* para los datos de *Imagenette* e *ImageWoof* de la Sección 4.2. Con cada uno de los extractores de características se construirá un clasificador auto-supervisado tal y como se detalló en la implementación de la Sección 3.5. En la figura 4.9 se muestra el tiempo necesario para entrenar las capas clasificadoras de cada arquitectura sobre las salidas congeladas de los extractores de características auto-supervisados. Debido a que los parámetros de las capas extractoras de características no se entrenan y modifican, solo se tiene

Resultados

que calcular el gradiente para los parámetros de las capas clasificadoras. Esto hace que el tiempo de entrenamiento sea más corto pro-

porcionalmente al obtenido en la clasificación supervisada (ver figura 4.1). No obstante, si se tiene también en cuenta el tiempo necesario para preentrenar de manera auto-supervisada los extractores de características con *SimSiam*, el tiempo total para el entrenamiento de la clasificación auto-supervisada es bastante superior al de la supervisada. Además, el preentrenamiento auto-supervisado con *SimSiam* necesita un mayor número de recursos de computación durante el preentrenamiento, al trabajar con 2 entradas de manera simultánea con las redes siamesas.

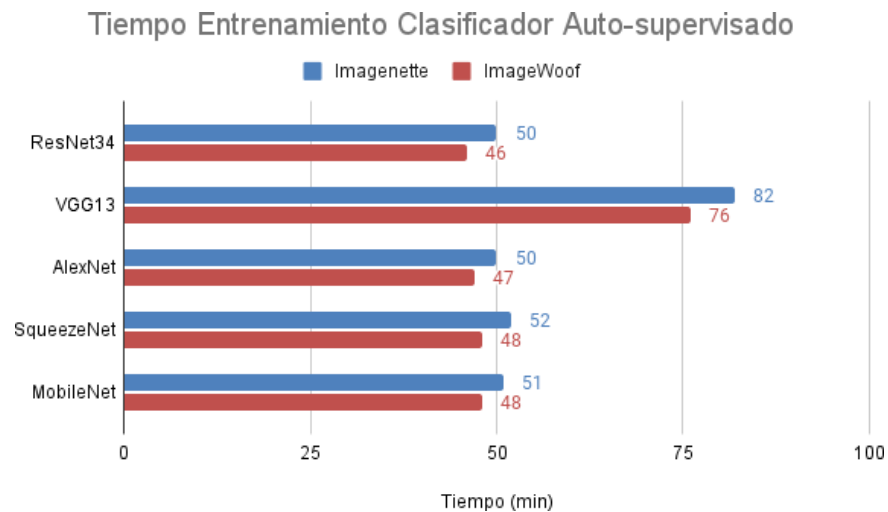


Figura 4.9: Tiempos de entrenamiento para la clasificación auto-supervisada por arquitectura en minutos. Se muestra los casos con *Imagenette* e *ImageWoof*.

En total se han entrenado 20 clasificadores auto-supervisados para la clasificación de *Imagenette* y otros 20 para la clasificación de *ImageWoof*. Dentro de estos 20 clasificadores se han entrenado 4 para cada arquitectura, variando en cada uno el número de épocas de preentrenamiento auto-supervisado con *SimSiam* de los extractores de características (100, 200, 400 y 800 épocas). En la figura 4.10 se pueden ver las *accuracies* top-1 obtenidas para cada caso.

En todos los casos, a excepción de *SqueezeNet*, a mayor número de épocas de preentrenamiento se obtienen *accuracies* mayores en los clasificadores auto-supervisados. Los mayores incrementos en la *accuracy* se producen en las 200 y 400 épocas de preentrenamiento. No obstante, de 400 a 800 épocas de preentrenamiento también se produce un incremento en la *accuracy*, aunque no tan significativo como en el preentrenamiento de las primeras 400 épocas. En la clasificación de *ImageWoof* se puede observar que los incrementos en la *accuracy* son mayores que en *Imagenette*. Por lo tanto, la clasificación auto-supervisada de clases similares, como en el caso de *ImageWoof*, obtiene un mayor beneficio de un mayor número de épocas de preentrenamiento con *SimSiam*. En el caso de *SqueezeNet1.1* se obtiene en ambos casos una mejor *accuracy* con 200 épocas de preentrenamiento que con 100, aunque a partir de más de 200 épocas de preentrenamiento no se obtiene un incremento significativo en la *accuracy* obtenida.

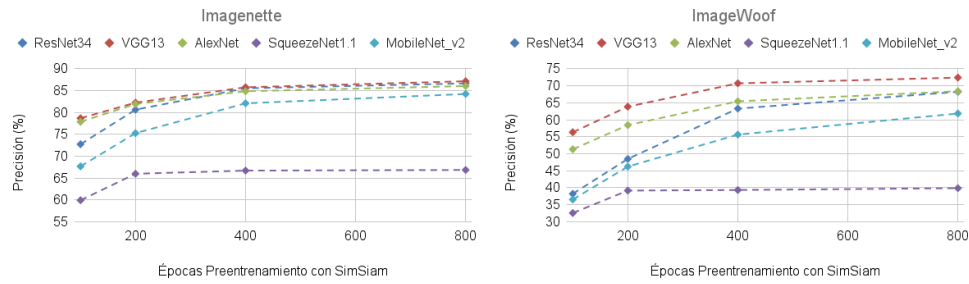


Figura 4.10: Efecto del número de épocas de preentrenamiento de los extractores de características con *SimSiam* en la *accuracy* top-1 (%) de los clasificadores auto-supervisados. A la izquierda la clasificación de *Imagenette* y a la derecha para la clasificación de *ImageWoof*.

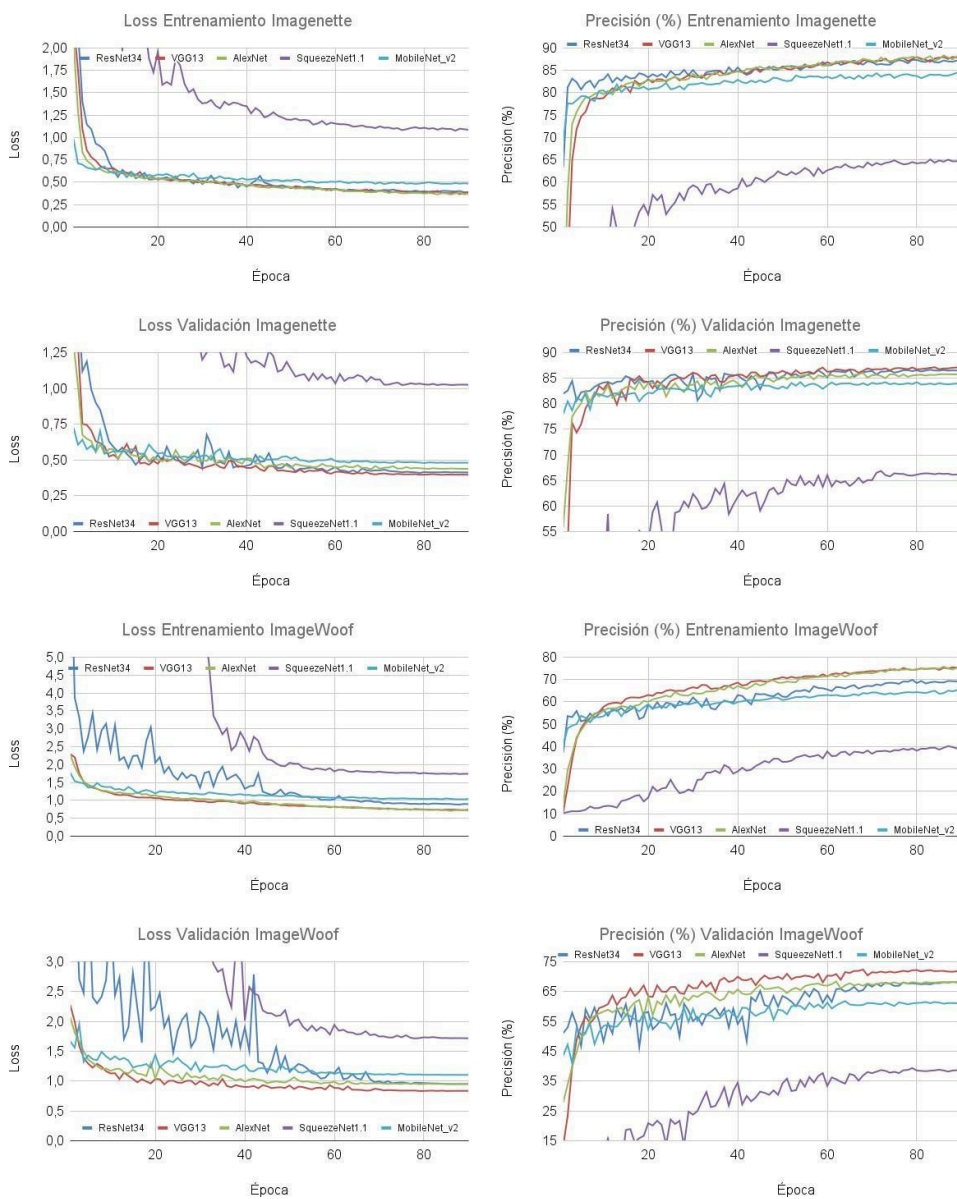


Figura 4.11: Gráficas de la *loss* y la *accuracy* del entrenamiento y la validación en la clasificación auto-supervisada de *Imagenette* e *ImageWoof* con 800 épocas de preentrenamiento con *SimSiam*.

En la figura 4.11 se puede ver la evolución de la *loss* y la *accuracy* para los clasificadores auto-supervisados preentrenados durante 800 épocas con *SimSiam*. En todos los casos la *loss* y la *accuracy* convergen de manera mucho más rápida que en la clasificación supervisada. Esto es debido a que al tener los extractores de características preentrenados, estos ya conocen buenas representaciones de los datos. De manera que con pocas épocas de entrenamiento, las capas clasificadoras son capaces de converger. Por ejemplo, en *Imagenette* todas las redes alcanzan una *accuracy* mayor a un 80 % con 10 épocas de entrenamiento, a excepción de *SqueezeNet*. En el caso de *ImageWoof* todas las redes alcanzan una *accuracy* mayor a un 55 % con 20 épocas de entrenamiento, a excepción de *SqueezeNet*.

4.3.1. Comparativa de la clasificación supervisada y autosupervisada

En la tabla 4.3 se muestran la *accuracy* top-1 y top-5 obtenidas con los clasificadores auto-supervisados. También se indica la diferencia en cada *accuracy* entre utilizar clasificadores auto-supervisados preentrenados con *SimSiam* y de manera supervisada. A pesar de que la clasificación supervisada de imágenes obtiene mejores resultados, la clasificación auto-supervisada no contrastiva de imágenes es capaz de obtener buenos resultados, siendo estos mejores a mayor número de parámetros y dimensión de las capas convolucionales, como se puede comprobar con VGG13 y *ResNet34*. Si solo se observan las *accuracy* top-1 y top-5 obtenidas con cada clasificador auto-supervisado, VGG13 obtiene otra vez los mejores resultados para las 100, 200, 400 y 800 épocas de preentrenamiento. Seguida de *ResNet34*, *AlexNet*, *MobileNetV2* y *SqueezeNet1.1*. Este es el mismo orden que se obtuvo en la *accuracy* del kNN durante el preentrenamiento y que se puede observar en la tabla 4.2, lo cual es congruente al ser un evaluador de las representaciones aprendidas.

La arquitectura que obtiene una menor diferencia entre los dos tipos de aprendizaje es *AlexNet*. Le siguen en orden de menor a mayor diferencia las arquitecturas VGG13, *ResNet34*, *MobileNetV2* y *SqueezeNet1.1*. Es notable la diferencia que se produce en las arquitecturas móviles como *MobileNetV2* y *SqueezeNet*. Esto es principalmente a que son las arquitecturas más ligeras y al diseño de sus capas convolucionales con un menor número de parámetros. Esto hace que no sean las arquitecturas más adecuadas para la extracción de características de manera auto-supervisada, aunque sí que sean adecuadas para la clasificación. En concreto, *SqueezeNet1.1* obtiene los peores resultados con un gran margen de diferencia. Esto se puede deber a que además de ser la arquitectura con menor número de parámetros, utiliza como capa clasificadora una capa convolucional. Mientras, en el resto de las CNNs se utiliza como clasificadores capas MLP, que son capas con un número mucho mayor de parámetros que una convolucional. La diferencia en la *accuracy* entre los distintos tipos de aprendizaje es mayor en *ImageWoof* que en *Imagenette* para todas las arquitecturas. Esto es debido a que los extractores de características preentrenados con

SimSiam tienen un mayor problema para separar y agrupar los datos de distintas clases similares (ver las figuras 4.7 y 4.8), dificultando la clasificación realizada posteriormente con las últimas capas.

4.3. RESULTADOS DE LA CLASIFICACIÓN AUTO-SUPERVISADA

Red	Imagenette				ImageWoof			
	Top-1	Dif.	Top-5	Dif.	Top-1	Dif.	Top-5	Dif.
<i>ResNet34</i> Supervisada	91	0	99, 44	0	85, 08	0	98, 34	0
<i>ResNet34</i> Auto-supervisada 100 épocas	72, 73	18, 27	97, 12	2, 32	38, 22	46, 86	83, 66	14, 68
<i>ResNet34</i> Auto-supervisada 200 épocas	80, 63	10, 37	98, 03	1, 41	48, 48	36, 6	89, 46	8, 88
<i>ResNet34</i> Auto-supervisada 400 épocas	85, 52	5, 48	99	0, 44	63, 27	21, 81	94, 57	3, 77
<i>ResNet34</i> Auto-supervisada 800 épocas	86, 64	4, 36	99, 13	0, 31	68, 21	16, 87	95, 69	2, 65
VGG13 Supervisada	91, 84	0	99, 46	0	85, 28	0	98, 57	0
VGG13 Auto-supervisada 100 épocas	<u>78, 67</u>	13, 17	<u>97, 98</u>	1, 48	<u>56, 32</u>	28, 96	<u>93</u>	<u>5, 57</u>
VGG13 Auto-supervisada 200 épocas	<u>82, 24</u>	9, 6	<u>98, 49</u>	0, 97	<u>63, 83</u>	21, 45	<u>94, 47</u>	<u>4, 1</u>
VGG13 Auto-supervisada 400 épocas	<u>85, 75</u>	6, 09	<u>99, 11</u>	0, 35	<u>70, 7</u>	14, 58	<u>96</u>	<u>2, 57</u>
VGG13 Auto-supervisada 800 épocas	<u>87, 13</u>	4, 71	<u>99, 23</u>	0, 23	<u>72, 38</u>	12, 9	<u>96, 25</u>	2, 32
<i>AlexNet</i> Supervisada	88, 81	0	99, 13	0	78, 41	0	97, 68	0
<i>AlexNet</i> Auto-supervisada 100 épocas	77, 86	<u>10, 95</u>	97, 7	<u>1, 43</u>	51, 31	<u>27, 1</u>	91, 6	6, 08
<i>AlexNet</i> Auto-supervisada 200 épocas	81, 93	<u>6, 88</u>	98, 49	<u>0, 64</u>	58, 41	<u>20</u>	93, 23	4, 45
<i>AlexNet</i> Auto-supervisada 400 épocas	84, 84	<u>3, 97</u>	98, 85	<u>0, 28</u>	65, 41	<u>13</u>	94, 75	2, 93
<i>Alexnet</i> Auto-supervisada 800 épocas	86, 03	<u>2, 78</u>	99, 1	<u>0, 03</u>	68, 38	<u>10, 03</u>	95, 72	<u>1, 96</u>
<i>SqueezeNet1.1</i> Supervisada	87, 79	0	99	0	80, 52	0	98, 32	0
<i>SqueezeNet1.1</i> Auto-supervisada 100 épocas	59, 92	27, 87	92, 68	6, 32	32, 57	47, 95	82, 43	15, 89
<i>SqueezeNet1.1</i> Auto-supervisada 200 épocas	65, 98	21, 81	95, 77	3, 23	39, 17	41, 35	86, 76	11, 56
<i>SqueezeNet1.1</i> Auto-supervisada 400 épocas	66, 7	21, 09	95, 89	3, 11	39, 34	41, 18	87, 35	10, 97
<i>SqueezeNet1.1</i> Auto-supervisada 800 épocas	66, 85	20, 94	96	3	39, 88	40, 64	87, 7	10, 62
<i>MobileNetV2</i> Supervisada	90, 39	0	99, 28	0	82, 92	0	98, 44	0
<i>MobileNetV2</i> Auto-supervisada 100 épocas	67, 66	22, 73	95, 92	3, 36	36, 54	46, 38	83, 48	14, 96
<i>MobileNetV2</i> Auto-supervisada 200 épocas	75, 26	15, 13	98, 08	1, 2	46, 22	36, 7	88, 95	9, 49
<i>MobileNetV2</i> Auto-supervisada 400 épocas	82, 08	8, 31	98, 7	0, 58	55, 63	27, 29	92, 95	5, 49
<i>MobileNetV2</i> Auto-supervisada 800 épocas	84, 22	6, 17	98, 85	0, 43	61, 82	21, 1	94, 68	3, 76

Tabla 4.3: Accuracy top-1 y top-5 (%) de cada clasificador auto-supervisado y su diferencia (%) con su correspondiente clasificador supervisado. Se muestra los resultados obtenidos para *Imagenette* e *ImageWoof* y se subrayan las precisiones más altas y las menores diferencias.

En este Trabajo de Fin de Grado se ha realizado un estudio e implementación de la **clasificación auto-supervisada no contrastiva de imágenes**, con el objetivo de poder comparar los resultados obtenidos con la clasificación supervisada tradicional. Para ello, se han estudiado e introducido los fundamentos de las redes neuronales, en concreto de las redes neuronales convolucionales profundas. Se han estudiado las diferentes capas que las forman y las principales arquitecturas que se usan en la actualidad, en concreto **ResNet** , **VGG**, **AlexNet** , **MobileNet** y **SqueezeNet**.

Una vez estudiadas las arquitecturas de las redes convolucionales, se han estudiado los diferentes tipos de aprendizaje, en particular el aprendizaje auto-supervisado. El aprendizaje auto-supervisado intenta extraer representaciones útiles a partir de datos no etiquetados a través de una tarea pretexto, con el objetivo de poder transferir este aprendizaje a otras tareas supervisadas. Esto es debido a que el etiquetado de bases de datos no anotadas es un proceso manual muy costoso y que requiere mucho tiempo y a veces conocimiento profesional. En concreto, se ha presentado y estudiado el método de aprendizaje auto-supervisado no contrastivo **SimSiam**.

Se han implementado y entrenado de manera supervisada las distintas redes convolucionales con **Pytorch**. Para ello se han utilizado técnicas para reducir el *overfitting* como puede ser el *weight decay*, *momentum*, *dropout* y *data augmentation* de los datos. En segundo lugar, se ha implementado *SimSiam* con *Pytorch* y se ha adecuado para cada red convolucional, ya que cada red tiene un tamaño diferente para su vector de características. En tercer lugar, se han entrenado los clasificadores auto-supervisados no contrastivos utilizando los extractores de características preentrenados con *SimSiam* y congelando sus pesos. Para los problemas de clasificación se ha trabajado sobre dos bases de datos diferentes, **Imagenette** e **ImageWoof**. La primera formada por 10 clases fácilmente distinguibles, mientras que en la segunda se tienen 10 clases menos distinguibles entre sí.

A continuación, se han presentado los resultados obtenidos para la clasificación supervisada, la clasificación auto-supervisada y el preentrenamiento con *SimSiam*. La clasificación supervisada obtiene mejores resultados que la clasificación auto-supervisada en todas las arquitecturas. A pesar de ello, la clasificación auto-supervisada obtiene buenos resultados y estos son mejores a mayor número de parámetros y complejidad de las capas convolucionales extractoras de características de las arquitec-

turas, destacando el caso de VGG y *ResNet*. Además, el preentrenamiento con *SimSiam* requiere de más tiempo y de un mayor gasto de recursos de computación que el entrenamiento supervisado. También se ha mostrado el efecto del número de épocas de preentrenamiento con *SimSiam* en la *accuracy* de cada arquitectura. Siendo los incrementos más notables en la *accuracy* durante las primeras 400 épocas de preentrenamiento.

Por último, se ha podido comprobar que gracias al preentrenamiento con *SimSiam*, los extractores de características de las redes son capaces de aprender buenas representaciones de los datos no etiquetados, siendo estas mejores a mayor número de parámetros de las capas extractoras. Este aprendizaje podrá ser transferido a otras redes para realizar otras tareas supervisadas posteriores, con el objetivo de facilitar el entrenamiento, mejorar los resultados y reducir el número de imágenes etiquetadas necesarias para el entrenamiento.

5.1. Trabajo futuro

A partir de los resultados obtenidos durante el trabajo, se propone las siguientes posibles tareas futuras:

1. – Debido al alto coste de etiquetar los datos, se propone entrenar los clasificadores supervisados y auto-supervisados con solo el 1 % y 10 % de las etiquetas de *Imagenette* e *ImageWoof*, para poder ver como afecta el número de datos etiquetados a los resultados obtenidos por ambos clasificadores.
2. – Entrenar distintos clasificadores auto-supervisados y supervisados con diferentes arquitecturas más pesadas como *ResNet50* y VGG16 para el caso de 1000 clases de *ImageNet*, y comparar los resultados obtenidos.
3. – Utilizar los extractores de características preentrenados con *SimSiam* para hacer *transfer learning* en otras tareas supervisadas como la detección de objetos.
4. – Implementar un método de aprendizaje contrastivo como *SimCLR* y comparar los clasificadores no contrastivos con los contrastivos.
5. – Utilizar el aprendizaje auto-supervisado no contrastivo en otros conjuntos de datos distintos como puede ser el audio, vídeo, sonido o el lenguaje natural.

BIBLIOGRAFÍA

- [1] X. Chen and K. He, "Exploring simple siamese representation learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15750–15758, 2021.
- [2] W. S. McCulloch and W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, 1943.
- [3] F. Rosenblatt, "The perceptron: a probabilistic model for information storage and organization in the brain.," *Psychological review*, vol. 65, no. 6, pp. 386–408, 1958.
- [4] S. L. Brunton and J. N. Kutz, *Neural Networks and Deep Learning*, p. 195–226. Cambridge University Press, 2019.
- [5] X. Wu, V. Kumar, Q. Ross, J. Ghosh, Q. Yang, H. Motoda, G. McLachlan, A. Ng, B. Liu, P. Yu, Z. Zhou, M. Steinbach, D. Hand, and D. Steinberg, "Top 10 algorithms in data mining," *Knowledge and Information Systems*, vol. 14, no. 1, pp. 1–37, 2008.
- [6] T. Alkarakatly, S. Eidhah, M. Al-Sarawani, A. Al-Sobhi, and M. Bilal, "Skin lesions identification using deep convolutional neural network," in *2019 International Conference on Advances in the Emerging Computing Technologies (AECT)*, pp. 1–5, 2020.
- [7] P. Kamencay, M. Benco, T. Mizdos, and R. Radil, "A new method for face recognition using convolutional neural network," *Advances in Electrical and Electronic Engineering*, vol. 15, no. 4, pp. 663–672, 2017.
- [8] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989.
- [9] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009.
- [10] N. Komodakis and S. Gidaris, "Unsupervised representation learning by predicting image rotations," in *International Conference on Learning Representations (ICLR)*, 2018.
- [11] C. Doersch, A. Gupta, and A. A. Efros, "Unsupervised visual representation learning by context prediction," in *Proceedings of the IEEE international conference on computer vision*, pp. 1422–1430, 2015.
- [12] A. Kolesnikov, X. Zhai, and L. Beyer, "Revisiting self-supervised visual representation learning," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 1920–1929, 2019.
- [13] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105,

Curran Associates, Inc., 2012.

- [14] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, vol. 9, pp. 249–256, PMLR, 2010.
- [15] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, pp. 1026–1034, 2015.
- [16] D. Masters and C. Luschi, "Revisiting small batch training for deep neural networks," *arXiv preprint: 1804.07612*, 2018.
- [17] P. Goyal, P. Dollár, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He, "Accurate, large minibatch sgd: Training imagenet in 1 hour," *arXiv preprint: 1706.02677*, 2017.
- [18] A. Krogh and J. Hertz, "A simple weight decay can improve generalization," in *Advances in Neural Information Processing Systems*, vol. 4, pp. 950–957, 1991.
- [19] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *Proceedings of the 30th International Conference on Machine Learning*, vol. 28, pp. 1139–1147, PMLR, 2013.
- [20] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [21] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proceedings of the 32nd International Conference on Machine Learning*, vol. 37, pp. 448–456, PMLR, 2015.
- [22] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint: 1409.1556*, 2014.
- [23] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- [24] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "Squeeze-net: Alexnet-level accuracy with 50x fewer parameters and <0.5 mb model size," *arXiv preprint: 1602.07360*, 2016.
- [25] R. D. Rakshit, D. R. Kisku, P. Gupta, and J. K. Sing, "Cross-resolution face identification using deep-convolutional neural network," *Multimedia Tools and Applications*, vol. 80, no. 14, pp. 20733–20758, 2021.
- [26] L. Liao, S. Li, Y. Che, W. Shi, and X. Wang, "Lithography hotspot detection method based on transfer learning using pre-trained deep convolutional neural network," *Applied Sciences*, vol. 12, no. 4, 2022.
- [27] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and

H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint: 1704.04861*, 2017.

- [28] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510–4520, 2018.
- [29] M. Noroozi and P. Favaro, "Unsupervised learning of visual representations by solving jigsaw puzzles," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VI*, pp. 69–84, 2016.
- [30] R. Zhang, P. Isola, and A. A. Efros, "Colorful image colorization," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part III* 14, pp. 649–666, 2016.
- [31] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*, pp. 1597–1607, 2020.
- [32] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* 32, pp. 8024–8035, Curran Associates, Inc., 2019.
- [33] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 11, pp. 2579–2605, 2008.

Appendices

| A

Para realizar los diferentes experimentos se ha adaptado el código original de *SimSiam* creado por *Facebook Research* ¹ para cada una de las redes neuronales convolucionales usadas. Se ha desarrollado usando Python y la librería *Pytorch*. El código de todo el proyecto se puede encontrar en <https://github.com/FLerma9/SimSiam>. Las principales funcionalidades se pueden encontrar en los archivos:

1. – *SupervisedClassifier.ipynb*: permite entrenar de manera supervisada las redes neuronales convolucionales *ResNet34*, *VGG13*, *AlexNet*, *MobileNetV2* y *SqueezeNet1.1*. Podremos indicar qué hiperparámetros usar como la arquitectura, épocas, base de datos, *learning rate*, *weight decay*, *learning rate scheduler*, *momentum* y el *batch size*. De manera general se usa el optimizador SGD y *Cross-Entropy loss*.
2. – *SimSiamTrainer.ipynb*: permite entrenar los extractores de características de las redes convolucionales nombradas con *SimSiam*. Utiliza la clase de *SimSiam* adaptada que se puede consultar en el apartado A.1 de este apéndice. Podremos indicar el *batch size*, número de épocas, *learning rate* base, *momentum* y *weight decay*. También realiza como validación una clasificación kNN con $k = 1$.
3. – *TSNE_Features.ipynb*: permite proyectar en un plano 2D mediante el algoritmo t-SNE, implementado en la librería de Python *sklearn*, las salidas de los *encoders* preentrenados con *SimSiam*.
4. – *SimSiamClassifier.ipynb*: permite realizar el entrenamiento de los clasificadores auto-supervisados. Para poder realizar la clasificación, tendremos que indicar al programa el *path* del modelo preentrenado con *SimSiam*. Permite al usuario modificar el *batch size*, número de épocas, base de datos, *learning rate*, *momentum* y *weight decay*.

A.1. Clase *SimSiam* adaptada

Como ya se ha indicado, se ha modificado la implementación original de *SimSiam* para que esta admitiese otros *backbones* de diferentes redes neuronales convolucionales. En concreto se ha implementado para las arquitecturas convolucionales *ResNet34*, *VGG13*, *AlexNet*, *MobileNetV2* y *SqueezeNet1.1*. El código se encuentra en el fichero *simsiam/builder.py* del repositorio GitHub del proyecto. A continuación se muestra el código Python desarrollado para la clase del proyector MLP, el predictor MLP y finalmente la de *SimSiam*:

¹ Repositorio del código original de *SimSiam*, <https://github.com/facebookresearch/simsiam>

Código A.1: Código de la clase del proyector MLP de *SimSiam*

```

1  from torch import nn
2  from torchvision.models import vgg13, resnet34, alexnet, squeezenet1_1, mobilenet_v2
3
4  class projector_MLP(nn.Module):
5      def init      (self, in_dim, out_dim=2048):
6          super(). init      ()
7          hidden_dim = 2048
8          self.fc1 = nn.Sequential(
9              nn.Linear(in_dim, hidden_dim),
10             nn.BatchNorm1d(hidden_dim),
11             nn.ReLU(inplace=True))
12         self.fc2 = nn.Sequential( nn.Linear(hidden_dim,
13             hidden_dim),
14             nn.BatchNorm1d(hidden_dim),
15             nn.ReLU(inplace=True))
16         self.fc3 = nn.Sequential( nn.Linear(hidden_dim,
17             out_dim), nn.BatchNorm1d(out_dim,
18             affine=False))
19
20
21     def forward(self, x):
22         x = self.fc1(x) x
23         = self.fc2(x) x =
24         self.fc3(x)
25         return x

```

Código A.2: Código de la clase del predictor MLP de *SimSiam*

```

1  class predictor_MLP(nn.Module):
2      def      init      (self,
3          in_dim=2048): super().
4          init      () hidden_dim =
5          int(in_dim / 4) self.fc1 =
6          nn.Sequential(
7              nn.Linear(in_dim, hidden_dim),
8              nn.BatchNorm1d(hidden_dim),
9              nn.ReLU(inplace=True))
10         self.fc2 = nn.Linear(hidden_dim, in_dim)
11
12     def forward(self, x):
13         x = self.fc1(x) x
14         = self.fc2(x)
15         return x

```

Código A.3: Código de la clase de *SimSiam*

```

1 class SimSiam(nn.Module):
2     def init      (self, arch='resnet34', dimension=2048):
3         super(SimSiam, self).      init
4         () self.arch = arch
5         #Definimos el back de SimSiam con la arquitectura que queremos
6         self.encoder = SimSiam.get_backbone(arch)
7
8
9         #Dependiendo de la arquitectura de la red la entrada del proyector MLP varía #Quitamos las
10        capas clasificadoras del backbone y añadimos el proyector
11        if 'resnet' in arch:
12            out_dim = self.encoder.fc.weight.shape[1] self.encoder.fc =
13            proyector_MLP(out_dim, dimension)
14        elif 'vgg' in arch:
15            out_dim = self.encoder.classifier[0].weight.shape[1]
16            self.encoder.classifier = proyector_MLP(out_dim, dimension)
17        elif 'squeezenet' in arch:
18            #Only if 224x224 input img
19            out_dim = 13*13*512
20            self.encoder.classifier = proyector_MLP(out_dim, dimension)
21        elif 'mobilenet_v2' in arch:
22            out_dim = self.encoder.classifier[1].weight.shape[1]
23            self.encoder.classifier = proyector_MLP(out_dim, dimension)
24        elif 'alexnet' in arch:
25            out_dim = self.encoder.classifier[1].weight.shape[1]
26            self.encoder.classifier = proyector_MLP(out_dim, dimension)
27            self.predictor = predictor_MLP(dimension)
28
29
30
31        #Definimos las arquitecturas que vamos a implementar
32        @staticmethod
33        def get_backbone(backbone_name):
34            return {'resnet34': resnet34(), 'mobilenet_v2':
35                mobilenet_v2(), 'squeezenet1_1':
36                squeezenet1_1(), 'vgg13': vgg13(),
37                'alexnet': alexnet()}[backbone_name]
38
39
40        def forward(self, x1, x2):
41            #Si es squeezenet tendremos que transformar los datos en un tensor de una columna
42            if self.arch == 'squeezenet1_1': x1 =
43                self.encoder.features(x1) x2 =
44                self.encoder.features(x2) x1 =
45                x1.view(x1.size(0), -1)
46                x2 = x2.view(x2.size(0), -1) z1 =
47                self.encoder.classifier(x1) z2 =
48                self.encoder.classifier(x2)
49            else:
50                z1 = self.encoder(x1)
51                z2 = self.encoder(x2)

```

A.2. Código *data augmentation*

A continuación se muestra el código responsable de cada *data augmentation* realizado con los datos:

Código A.4: Código de las transformaciones de las imágenes de entrenamiento en la clasificación supervisada.

```
1 transform_train = transforms.Compose(  
2     [transforms.RandomResizedCrop(224),  
3     transforms.RandomHorizontalFlip(),  
4     transforms.ColorJitter(0.4,0.4,0.4,0.1),  
5     transforms.ToTensor(),  
6     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Código A.5: Código de las transformaciones de las imágenes de validación en la clasificación supervisada y auto-supervisada.

```
1 transform_val = transforms.Compose(  
2     [transforms.Resize((256,256)),  
3     transforms.CenterCrop(224),  
4     transforms.ToTensor(),  
5     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Código A.6: Código de las transformaciones de las imágenes de entrenamiento de SimSiam.

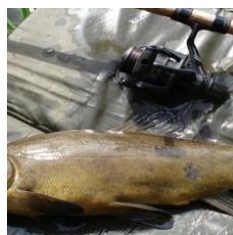
```
1 transform_train = transforms.Compose([ transforms.RandomResizedCrop(224,  
2     scale=(0.2, 1.)),  
3     transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.1)], p=0.8),  
4     transforms.RandomGrayscale(p=0.2),  
5     transforms.RandomApply([simiam.loader.GaussianBlur([.1, 2.])], p=0.5),  
6     transforms.RandomHorizontalFlip(),  
7     transforms.ToTensor(),  
8     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])])
```

Código A.7: Código de las transformaciones de las imágenes de entrenamiento en la clasificación auto-supervisada.

```
1 transform_train = transforms.Compose(  
2     [transforms.RandomResizedCrop(224),  
3     transforms.RandomHorizontalFlip(),  
4     transforms.ToTensor(),  
5     transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])  
6 ])
```



(a) Original



(b) Recorte Aleatorio



(c) Color



(d) Blur



(e) Volteo Horizontal



(f) Blanco y Negro

Figura A.1: Esta figura muestra las 5 posibles modificaciones que se pueden realizar a una misma imagen en SimSiam. Se muestra la imagen original, un recorte aleatorio de ella, una modificación del color, una difuminación de la imagen (*blur*), un volteo horizontal y en blanco y negro.

A.3. Instalación

A continuación se muestran los comandos para instalar mediante Conda la librería *Pytorch* y *Neptune AI*:

Código A.8: Comando de instalación de *Pytorch*.

```
1 conda install pytorch torchvision torchaudio pytorch-cuda=11.8 -c pytorch -c nvidia
```

Código A.9: Comando de instalación de *Neptune*.

```
1 conda install -c conda-forge neptune
```




Universidad Autónoma
de Madrid